KPR Institute of Engineering and Technology

Department of Artificial Intelligence and Data Science

ASSIGNMENT II – CASE STUDY (PROJECT 2: REGRESSION)

Predicting Final Student Grades from Academic, Family and Lifestyle Factors Using Exploratory Data Analysis and Deep Learning

Case Study: Exploratory Data Analysis and Visualization of a Real-World Dataset

Student Name	RITHI R
Roll Number / College ID	24AD091
Department	Artificial Intelligence and Data Science
Year / Semester	III Year / Semester V (Batch 2023–2027)
Course Code	U21ADP05
Course Title	Exploratory Data Analysis and Visualization
Assignment	Assignment II – Case Study (40 marks)
Academic Year	2026–2027 (Odd Semester)
Course Coordinator	Mr. Rushikesh Kadam
Report Date	19 October 2026

Abstract

Predicting a student's final grade early helps schools offer support in time. This study analyses the UCI Student Performance dataset (Portuguese course: 649 students, 32 input features, final grade G3 on a 0–20 scale). Exploratory analysis found no missing values, then used seven-stage visualisation, statistical tests, distribution fitting and PCA. Earlier grades dominated: G2 correlates with G3 at r = 0.92, while past failures, school and wish for higher education were the strongest non-grade factors. A multilayer perceptron regressor, tuned over 12 settings, reached test RMSE 1.33 (R^2 0.82) with earlier grades, but RMSE 2.93 (R^2 0.12) without them. A random forest matched or beat it, so predictions from lifestyle factors alone remain weak.

Keywords: exploratory data analysis, data visualization, student performance, regression, multilayer perceptron.

(Abstract: 114 words)

Table of Contents

1. Introduction and Objectives....7
1.1 Background....7
1.2 Problem Statement....7
1.3 Motivation....7
1.4 Objectives....7
1.5 Scope and Practical Relevance....7
2. Dataset Description....8
2.1 Dataset Name, Source and Licence....8
2.2 Why This Dataset Was Chosen....8
2.3 Verified Structure of the Dataset....8
2.4 Description of the Fields....9
2.5 Basic Descriptive Statistics....10
3. Exploratory Data Analysis and Preprocessing....11
3.1 Loading and Inspecting the Data....11
3.2 Missing Values and Duplicates....11
3.3 Outlier Identification and Treatment....11
3.4 Cleaning, Transformation and Encoding....12
3.5 Two Experimental Settings....12
3.6 Train / Validation / Test Split and Leakage Prevention....12
3.7 Key Findings from EDA....13
4. Data Visualization....13
4.1 The Seven Stages of Visualization....13
4.2 Visualization Design Choices (Refine stage)....13
4.3 Interactive Visualizations....14
4.4 Visualization Gallery with Explanations....14
5. Statistical and Distribution-Based Analysis....23
5.1 Univariate Analysis....23
5.2 Probability Distributions....23
5.3 Bivariate Analysis....24
5.4 Multivariate Analysis....25
5.5 Summary of Statistical Findings....25
6. Deep Learning Model....25
6.1 Why a Multilayer Perceptron?....25
6.2 Inputs, Target and Pipeline....25
6.3 Architecture and Hyperparameters....26
6.4 Training, Validation and Testing Procedure....26
6.5 Basic Hyperparameter Tuning....26
6.6 Feature Analysis and Feature Importance....27
7. Model Evaluation and Result Visualization....28
7.1 Evaluation Metrics....28
7.2 Comparison with Baseline Models....29
7.3 Result Visualizations and Interpretation....29
7.4 Stability over 10 Random Splits....33
8. Conclusion, Challenges and Future Scope....33
8.1 What Was Learned from the Data....33
8.2 Model Performance....34
8.3 Challenges and Limitations....34

8.4 Future Scope 34
9. Literature Review and References 34
9.1 Short Literature Review 34
9.2 References (APA 7th edition) 35
10. Rubric Alignment 35
Appendix A – Complete Python Source Code 37
A.1 config.py – paths, seed, column groups, plot style 37
A.2 data_preprocessing.py – loading, cleaning, split, pipeline 37
A.3 eda.py – statistics and distribution fitting 38
A.4 visualizations.py – Figures 1–12 and interactive pages 39
A.5 train_model.py – tuning, training, baselines, repeated splits 42
A.6 evaluate_model.py – evaluation, Figures 13–20 43
A.7 diagrams.py – workflow and architecture diagrams 45
A.8 run_all.py – run the whole pipeline 45
Appendix B – GitHub Repository and Checklist 46
B.1 Repository Structure 46
B.2 README Outline 46
B.3 Commands to Publish 46
B.4 GitHub Submission Checklist 47
Appendix C – Final Checklist Against the Assignment Requirements 47

List of Figures

Diagram 1: End-to-end project workflow 11
Figure 1: Distribution of the final grade (G3) 14
Figure 2: Histograms of key numeric variables 15
Figure 3: Final grade by key factors 15
Figure 4: Share of IQR outliers per numeric feature 16
Figure 5: Continuous distribution: final grade G3 16
Figure 6: Discrete distributions: Poisson, negative binomial, Binomial 17
Figure 7: Spearman correlation heatmap 18
Figure 8: Mean final grade for each binary factor 19
Figure 9: Earlier grades against the final grade 20
Figure 10: PCA projection of numeric features 21
Figure 11: Pairplot of key variables 22
Figure 12: Mean G3 by study time and past failures 23
Diagram 2: MLP architecture of the final model (setting A) 26
Figure 13: Training vs validation loss 29
Figure 14: Training vs validation MAE 30
Figure 15: Predicted vs actual final grade 30
Figure 16: Error distribution 31
Figure 17: Residual plot 31
Figure 18: Permutation importance (top 10 features) 32
Figure 19: Hyperparameter search results 32
Figure 20: Stability over 10 random train/validation/test splits 33

List of Tables

Table 1: Dataset identification..... 8
Table 2: Verified facts about the loaded file..... 8
Table 3: All 32 input fields and the target..... 9
Table 4: Descriptive statistics of numeric features and the target..... 10
Table 5: Outlier summary..... 11
Table 6: Preprocessing applied to each group of features..... 12
Table 7: The two settings..... 12
Table 8: Data split (random_state = 42)..... 12
Table 9: Seven stages of visualization applied to the student grade dataset..... 13
Table 10: Distribution analysis results..... 23
Table 11: Fourteen features most strongly associated with G3..... 24
Table 12: Statistical findings in plain words..... 25
Table 13: Model inputs and output..... 25
Table 14: Final model configuration..... 26
Table 15: Best five configurations, setting A (sorted by validation RMSE)..... 27
Table 16: Best five configurations, setting B (sorted by validation RMSE)..... 27
Table 17: Top eight features by permutation importance, setting A (test set)..... 27
Table 18: Top eight features by permutation importance, setting B (test set)..... 27
Table 19: Regression metrics for setting A (with G1 and G2)..... 28
Table 20: Regression metrics for setting B (without G1 and G2)..... 28
Table 21: Test errors over 10 random splits (mean ± standard deviation)..... 33
Table 22: Rubric alignment..... 35
Table 23: Purpose of each folder and file..... 46
Table 24: Checklist before submitting the link..... 47
Table 25: Requirement checklist..... 47

List of Abbreviations

Abbreviation	Meaning
CO	Course outcome
CSV	Comma-separated values
EDA	Exploratory data analysis
G1, G2, G3	First-period, second-period and final grade (0–20)
IQR	Inter-quartile range
MAE	Mean absolute error
MLP	Multilayer perceptron
MSE	Mean squared error
PCA	Principal component analysis
R^2	Coefficient of determination
ReLU	Rectified linear unit
RMSE	Root mean squared error
UCI	University of California, Irvine (Machine Learning Repository)
VIF	Variance inflation factor

1. Introduction and Objectives

1.1 Background

Schools record a student's marks in every period together with details about the family, study habits and free-time activities. Exploratory Data Analysis (EDA), introduced by John Tukey (1977), means looking at such data through summaries and pictures before building a model, so that patterns, errors and surprises are found early. In this project, EDA and a deep-learning model are applied to the marks of secondary-school students in a Portuguese-language course.

1.2 Problem Statement

Given a student's background, study habits, lifestyle and (optionally) earlier-period marks, (a) which factors are most strongly associated with the **final grade G3**, and (b) how accurately can a deep-learning model **predict G3 as a number between 0 and 20**? Because the target is numeric, this is a **regression** problem. (The companion project in this series, on student dropout, was a classification problem.)

1.3 Motivation

- A teacher who sees a low predicted grade early can offer extra help before the final examination.
- It matters *how early* the prediction is made. A model that uses the marks of the first two periods is much easier to build than one that uses only background information. This report measures both situations separately.
- The data are real, mixed (numeric, ordinal, binary and categorical) and small enough to study carefully on a laptop.

1.4 Objectives

1. Select and verify a real-world dataset with at least 15 features (this one has 32 input features).
2. Perform data understanding, cleaning, outlier analysis, encoding, scaling and a train/validation/test split without data leakage.
3. Create more than five meaningful visualisations (12 EDA figures and 8 model figures) following all seven stages of visualisation.
4. Carry out univariate, bivariate and multivariate analysis and study continuous and discrete probability distributions.
5. Build and tune a multilayer perceptron regressor in two settings (with and without the earlier grades) and compare it with simple baselines.
6. Evaluate it with loss/MAE curves, predicted-versus-actual and error-distribution plots, residual analysis and feature importance.
7. Publish the work in a well-structured GitHub repository with a README.

1.5 Scope and Practical Relevance

The study covers one dataset from two schools in one region and a single course (Portuguese language). It is a learning project, not a deployed system. The workflow (clean, explore, visualise, model, evaluate) is the one a school analytics team would follow to build an early-warning tool for teachers.

2. Dataset Description

2.1 Dataset Name, Source and Licence

Table 1: Dataset identification

Item	Details
Name	Student Performance
Repository	UCI Machine Learning Repository (dataset id 320)
URL	https://archive.ics.uci.edu/dataset/320/student+performance
DOI	10.24432/C5TG7T
Creator	Paulo Cortez (University of Minho, Portugal)
Licence	Creative Commons Attribution 4.0 (CC BY 4.0)
Companion paper	Cortez & Silva (2008), Using data mining to predict secondary school student performance, FUBUTEC 2008, pp. 5–12
File used	student-por.csv (Portuguese-language course) saved as data/student-por.csv

Verification note. The UCI page lists 649 instances, 30 attributes plus the three grades G1, G2 and G3, no missing values, and the tasks classification and regression. The UCI page does not say which of its two files (Mathematics and Portuguese) holds the 649 rows; the Portuguese file used here contains exactly 649 rows, which matches. The UCI website could not be reached from the environment used for the analysis, so the CSV was obtained from public GitHub copies of the dataset. Two independent copies were loaded and compared and found identical (649 × 33). The original UCI file is semicolon-separated; the copies are comma-separated, and this is handled in the loading code. Anyone re-running the project can download the original from the UCI link and place it at data/student-por.csv (use sep=';' when loading the original).

2.2 Why This Dataset Was Chosen

- **More than 15 features:** 32 input fields plus a target.
- **Real-world relevance:** real students from two schools; the problem (predicting final marks) is a standard learning-analytics task.
- **Mixed data types:** ordinal ratings (1–5), counts, binary yes/no flags and categorical fields, so many chart types and statistical tests are meaningful.
- **Suitable for regression:** the target G3 is an integer from 0 to 20 and is treated as a numeric value; the UCI page lists regression as an associated task.
- **A teachable twist:** G1 and G2 are almost duplicates of G3, so the project can show how much a model depends on them by training with and without them.
- **Small and clean:** 649 rows run quickly on a laptop. The small size also forces honest evaluation (repeated splits).

2.3 Verified Structure of the Dataset

Table 2: Verified facts about the loaded file

Property	Value (measured on data/student-por.csv)
Rows (students)	649
Columns	33 (32 input features + target G3)
Missing cells	0

Property	Value (measured on data/student-por.csv)
Duplicate rows	0
Numeric / ordinal features	15 (age, parents' education, study time, failures, ratings, absences, G1, G2)
Binary (yes/no-type) features	13
Nominal features (>2 categories)	4 (Mjob, Fjob, reason, guardian)
Stored data types	integer: 16, text: 17
Target	G3 – final grade, integer 0–20 (regression)
G3 summary	mean 11.91, median 12, SD 3.23, min 0, max 19
Passing students (G3 ≥ 10)	549 of 649 (84.6%)

2.4 Description of the Fields

The descriptions below follow the attribute list on the UCI dataset page. Ordinal ratings (1–5) are treated as numbers in this report.

Table 3: All 32 input fields and the target

#	Field	Group	Meaning
1	school	Binary	Student's school (GP or MS)
2	sex	Binary	Sex (F or M)
3	age	Numeric	Age in years (15–22)
4	address	Binary	Home address type (U urban, R rural)
5	famsize	Binary	Family size (LE3 up to 3, GT3 more than 3)
6	Pstatus	Binary	Parents' cohabitation (T together, A apart)
7	Medu	Numeric	Mother's education (0–4)
8	Fedu	Numeric	Father's education (0–4)
9	Mjob	Nominal	Mother's job (category)
10	Fjob	Nominal	Father's job (category)
11	reason	Nominal	Reason for choosing the school (category)
12	guardian	Nominal	Student's guardian (mother, father, other)
13	traveltime	Numeric	Home-to-school travel time (1–4)
14	studytime	Numeric	Weekly study time (1–4)
15	failures	Numeric	Number of past class failures
16	schoolsup	Binary	Extra educational support from school
17	famsup	Binary	Family educational support
18	paid	Binary	Extra paid classes in the course
19	activities	Binary	Extra-curricular activities
20	nursery	Binary	Attended nursery school
21	higher	Binary	Wants to take higher education
22	internet	Binary	Internet access at home
23	romantic	Binary	In a romantic relationship
24	famrel	Numeric	Quality of family relationships (1–5)
25	freetime	Numeric	Free time after school (1–5)
26	goout	Numeric	Going out with friends (1–5)
27	Dalc	Numeric	Workday alcohol consumption (1–5)
28	Walc	Numeric	Weekend alcohol consumption (1–5)

#	Field	Group	Meaning
29	health	Numeric	Current health status (1–5)
30	absences	Numeric	Number of school absences
31	G1	Numeric	First-period grade (0–20)
32	G2	Numeric	Second-period grade (0–20)
33	G3	Target	Final grade (0–20) – the value to be predicted

2.5 Basic Descriptive Statistics

The table gives the mean, median, mode, standard deviation, quartiles and skewness of the 15 numeric/ordinal features and the target, computed on all 649 rows.

Table 4: Descriptive statistics of numeric features and the target

Feature	Mean	Median	Mode	Std	Min	Q1	Q3	Max	Skew
age	16.74	17.00	17.00	1.22	15.00	16.00	18.00	22.00	0.42
Medu	2.52	2.00	2.00	1.14	0.00	2.00	4.00	4.00	-0.03
Fedu	2.31	2.00	2.00	1.10	0.00	1.00	3.00	4.00	0.21
traveltime	1.57	1.00	1.00	0.75	1.00	1.00	2.00	4.00	1.25
studytime	1.93	2.00	2.00	0.83	1.00	1.00	2.00	4.00	0.70
failures	0.22	0.00	0.00	0.59	0.00	0.00	0.00	3.00	3.09
famrel	3.93	4.00	4.00	0.96	1.00	4.00	5.00	5.00	-1.11
freetime	3.18	3.00	3.00	1.05	1.00	3.00	4.00	5.00	-0.18
goout	3.19	3.00	3.00	1.18	1.00	2.00	4.00	5.00	-0.01
Dalc	1.50	1.00	1.00	0.93	1.00	1.00	2.00	5.00	2.14
Walc	2.28	2.00	1.00	1.28	1.00	1.00	3.00	5.00	0.64
health	3.54	4.00	5.00	1.45	1.00	2.00	5.00	5.00	-0.50
absences	3.66	2.00	0.00	4.64	0.00	0.00	6.00	32.00	2.02
G1	11.40	11.00	10.00	2.75	0.00	10.00	13.00	19.00	-0.00
G2	11.57	11.00	11.00	2.91	0.00	10.00	13.00	19.00	-0.36
G3	11.91	12.00	11.00	3.23	0.00	10.00	14.00	19.00	-0.91

3. Exploratory Data Analysis and Preprocessing

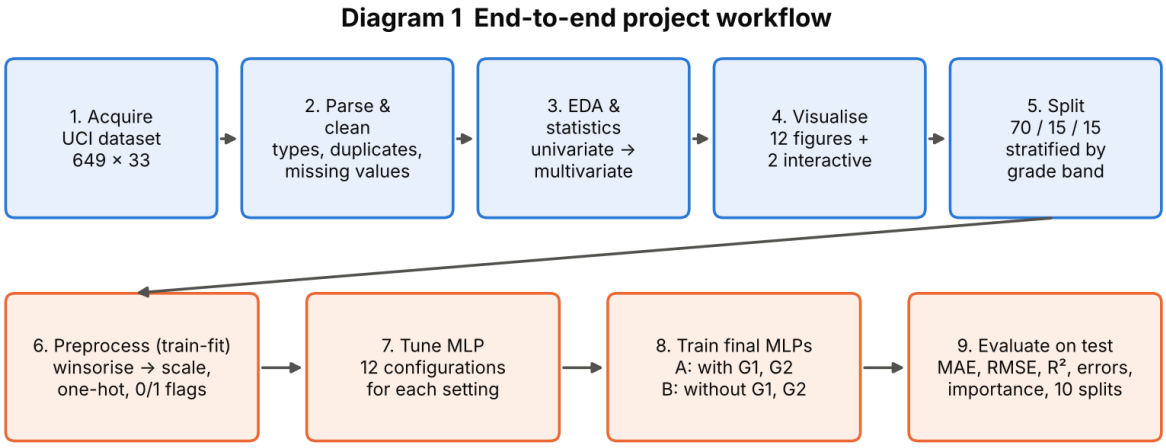


Diagram 1: End-to-end project workflow

Diagram 1 shows the order of work. Steps 1–5 are data understanding and visualisation; steps 6–9 are modelling. Each step is a separate Python module in the GitHub repository.

3.1 Loading and Inspecting the Data

The CSV is loaded with pandas (pd.read_csv). The shape is 649 rows × 33 columns. df.info(), df.describe() and value counts were used to check data types and ranges. All ordinal ratings are within their documented ranges.

3.2 Missing Values and Duplicates

Missing cells: **0**. Duplicate rows: **0**. The cleaning function therefore removed no rows. The check remains in the code so that the same script is safe on other data.

A hidden pattern: the zero grades. 15 students (2.3%) have a final grade of exactly 0, while the next lowest values are 1 and 5 (Figure 1). All 15 of them have **0 absences**, and their average second-period grade was 3.8 (only 7 had G2 = 0). A student with a decent G2 but G3 = 0 is more consistent with *not taking the final assessment* than with scoring zero marks. The dataset documentation does not explain this, so it is stated here only as a hypothesis. These rows are real and were not removed, but they turn out to be the largest source of prediction error (Chapter 7).

3.3 Outlier Identification and Treatment

Outliers were identified with the IQR rule and the $|z| > 3$ rule for numeric columns with more than five distinct values (Table 5, Figure 4).

Table 5: Outlier summary

Feature	Lower fence	Upper fence	IQR outliers	Share	|z|>3
G2	5.50	17.50	25	3.9%	7
absences	-9.00	15.00	21	3.2%	11
G1	5.50	17.50	16	2.5%	1
G3	4.00	20.00	16	2.5%	16
age	13.00	21.00	1	0.2%	3

Decision. Few rows are flagged (at most 3.9% in any column). The flagged values are real: very high absences (maximum 32) and the zero grades. No row was deleted. Inside the model pipeline each numeric feature is winsorised (clipped to its 1st and 99th training percentile) so that a few extreme values cannot dominate the scaling.

3.4 Cleaning, Transformation and Encoding

Table 6: Preprocessing applied to each group of features

Group	Features	Treatment	Reason
Numeric / ordinal	15 fields (age, ratings, absences, G1, G2, …)	Winsorise 1–99 % → StandardScaler (mean 0, SD 1)	Neural networks train poorly when inputs have very different scales
Nominal	4 fields (Mjob, Fjob, reason, guardian)	One-hot encoding	Categories have no order, so they must not be coded as 1, 2, 3
Binary	13 fields (school, sex, internet, higher, …)	Encoded as 0/1 (ordinal encoder fitted on training rows)	Two-level text must become numbers
Target	G3	Kept in grade points; standardised with the training mean/SD only while training; predictions converted back and clipped to 0–20	Makes training stable but reports errors in real grade points

After encoding, the 32 raw features become **45 model input columns** in setting A and **43** in setting B (see Section 3.5).

3.5 Two Experimental Settings

G1 and G2 are marks of earlier periods of the *same course*, so they are known only after those periods. To show how much they matter, two settings are studied throughout:

Table 7: The two settings

Setting	Inputs	Meaning in practice
A – with grades	All 32 features incl. G1, G2 (45 encoded columns)	Prediction after two periods of the course
B – without grades	30 features, no G1, G2 (43 encoded columns)	Early warning using only background and lifestyle information

3.6 Train / Validation / Test Split and Leakage Prevention

Table 8: Data split (random_state = 42)

Subset	Rows	Share	Purpose
Training	454	70%	Fit the scaler/encoder and learn the network weights
Validation	97	15%	Choose hyperparameters and stop training early
Test	98	15%	Final, one-time evaluation

Preventing data leakage. (1) The split is done before any statistic is learned. (2) The winsoriser, scaler and encoders are fitted on training rows only. (3) The test set is used only after all tuning decisions are final. (4) The split is stratified on three grade bands (below 10, 10–13, 14 and above) so that every subset has a similar grade mix. (5) The 649 rows are different students within this file, so no student appears twice.

Why also repeat the split 10 times? A test set of 98 students is small, and a few unusual students (such as zero-grade cases) can change an error figure a lot. Section 7.4 therefore also reports results over 10 random splits. The hyperparameters were chosen once, on the seed-42 validation set.

3.7 Key Findings from EDA

- 84.6% of students pass (G3 ≥ 10); the grade distribution is concentrated between 8 and 16 with a small spike at 0.
- The earlier grades G1 and G2 are almost the same quantity as G3 (Pearson r = 0.83 and 0.92).
- Past failures matter: mean G3 is 12.5 with no past failure versus 8.6 with one or more.
- Wanting higher education (mean G3 12.3 vs 8.8) and school (GP 12.6 vs MS 10.7) show large differences.
- Most lifestyle ratings show weak relationships with G3 (|Spearman ρ| below 0.25), and parents' education is the strongest of the family variables.
- Absences are heavily skewed: 37.6% of students have none, the mean is 3.7 and the maximum is 32.

4. Data Visualization

4.1 The Seven Stages of Visualization

The assignment sheet asks that the visualisations follow "all 7 stages of visualization" but does not list them. This report therefore states the framework it uses: the seven-stage visualisation process of Ben Fry (2007), *Visualizing Data* (O'Reilly): **Acquire, Parse, Filter, Mine, Represent, Refine, Interact**.

Table 9: Seven stages of visualization applied to the student grade dataset

Stage	Meaning	How it was applied here	Evidence
1. Acquire	Obtain the data	Selected UCI dataset 320, checked licence, rows and columns, and loaded data/student-por.csv	Sections 2.1, 2.3
2. Parse	Give the data structure and meaning	Assigned numeric, ordinal, binary and nominal roles; defined the target and the two settings	Sections 2.4, 3.4, 3.5
3. Filter	Remove what is not needed	Missing/duplicate checks; chose key variables per chart; kept only levels with enough students in heat-maps	Sections 3.2, 3.3
4. Mine	Find patterns with statistics	Spearman, Mann–Whitney, Kruskal–Wallis, VIF, PCA, distribution fitting	Chapter 5
5. Represent	Choose a basic visual form	Bar, histogram, boxplot, scatter, heat-map, pairplot, Q–Q, PCA, dot-and-interval chart matched to the data type	Figures 1–12
6. Refine	Improve clarity and design	Colour-blind-safe palette, consistent colours (blue pass, orange fail), direct labels, mean/median lines, jitter and transparency, sequential scales for ordered values	All figures
7. Interact	Let the viewer explore	Two Plotly pages with hover, zoom and legend filtering: outputs/eda/interactive_*.html*	Section 4.3

4.2 Visualization Design Choices (Refine stage)

Colours follow a fixed meaning: **blue = pass (or the main series), orange = fail (or the comparison series)**. Both colours come from a palette whose adjacent colours were checked for colour-vision-deficiency separation. Ordered quantities (mean grade, PCA colouring) use a single-hue or perceptually uniform sequential scale; the correlation heat-map uses a diverging scale centred at zero. Axes carry units, titles are numbered and no chart uses a dual axis.

4.3 Interactive Visualizations

Two interactive HTML pages are saved in outputs/eda/: **interactive_g2_vs_g3.html** (hover shows G1, failures, study time, absences, school and wish for higher education; clicking a legend entry hides or shows pass or fail) and **interactive_g3_by_failures.html** (box plots of G3 by past failures and school with all points). They open in any browser and complete stage 7.

4.4 Visualization Gallery with Explanations

Each figure is followed by a table stating its purpose, variables, chart type, why it was selected and the insight obtained, as the assignment requires. Figures 1–12 are EDA figures; model figures are in Chapter 7.

Fig. 1 Distribution of the final grade (G3)

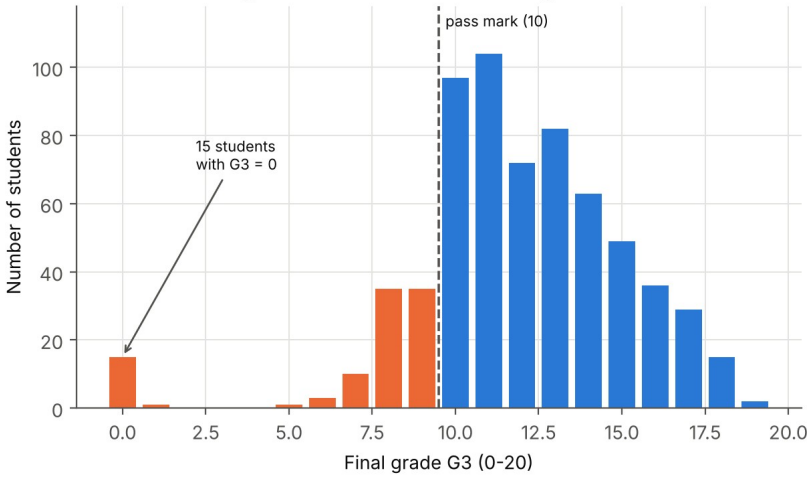


Figure 1: Distribution of the final grade (G3)

Aspect	Explanation
Purpose	Show how final grades are spread and how many students pass.
Variables	G3
Chart type	Bar chart (one bar per grade) with pass mark
Why selected	G3 is an integer, so one bar per value is more honest than binned bars; colouring the bars by pass/fail adds meaning.
Insight	Grades are concentrated between 8 and 16 (median 12, mean 11.9). 549 of 649 students (84.6%) reach the pass mark. A separate group of 15 students has G3 = 0, far below the rest.

Fig. 2 Histograms of key numeric variables

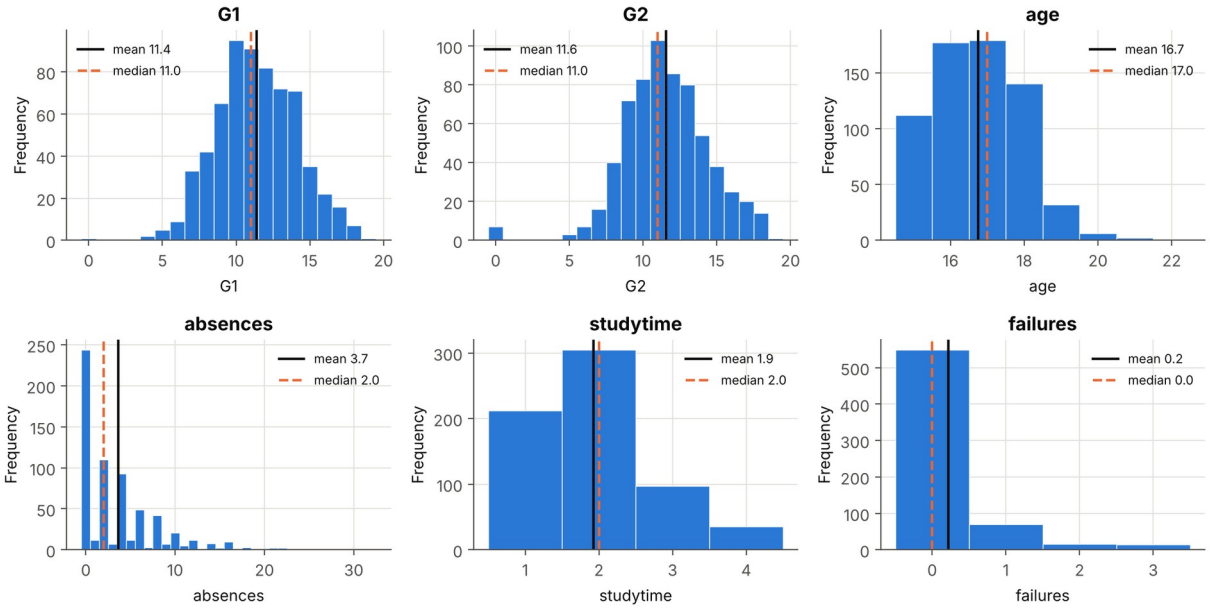


Figure 2: Histograms of key numeric variables

Aspect	Explanation
Purpose	Show the shape, centre and spread of six variables.
Variables	G1, G2, age, absences, studytime, failures
Chart type	Histograms with mean (solid) and median (dashed) lines
Why selected	Histograms are the standard univariate view; the gap between mean and median shows skewness.
Insight	G1 and G2 are bell-shaped around 11–12. Age is concentrated at 15–18. Absences are strongly right-skewed (median 2, mean 3.7, maximum 32). Most students have no past failure (mean 0.22); the most common study-time level is 2 (2–5 hours a week).

Fig. 3 Final grade (G3) by key factors

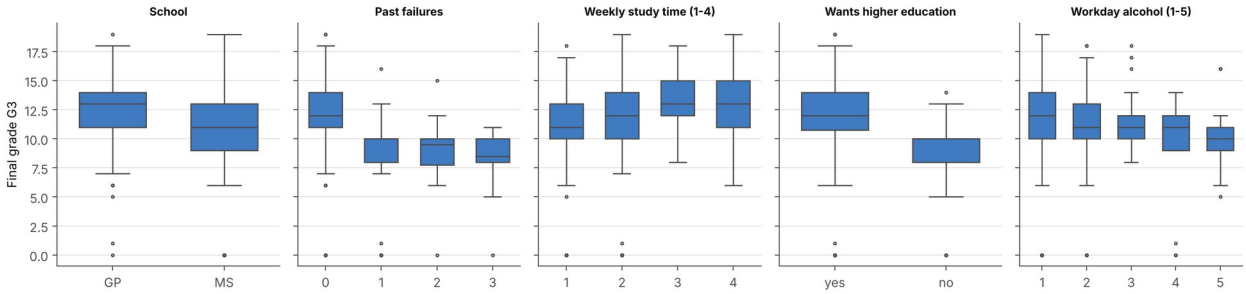


Figure 3: Final grade by key factors

Aspect	Explanation
Purpose	Compare the grade distribution across groups and reveal outliers.
Variables	G3 vs school, failures, studytime, higher, Dalc
Chart type	Grouped boxplots
Why selected	Boxplots compare medians, spread and outliers between groups compactly.
Insight	Median G3 is clearly lower for students with past failures (median 12 with none, 10 with one or two) and tends to rise with study time (median 11, 12, 13, 13 for levels 1–4). Students who do not want higher education and students of school MS have lower medians. Workday alcohol use shows a slight downward trend. The group means are: failures 0 → 12.5, 1 → 8.6; study time 1 → 10.8, 3 → 13.2.

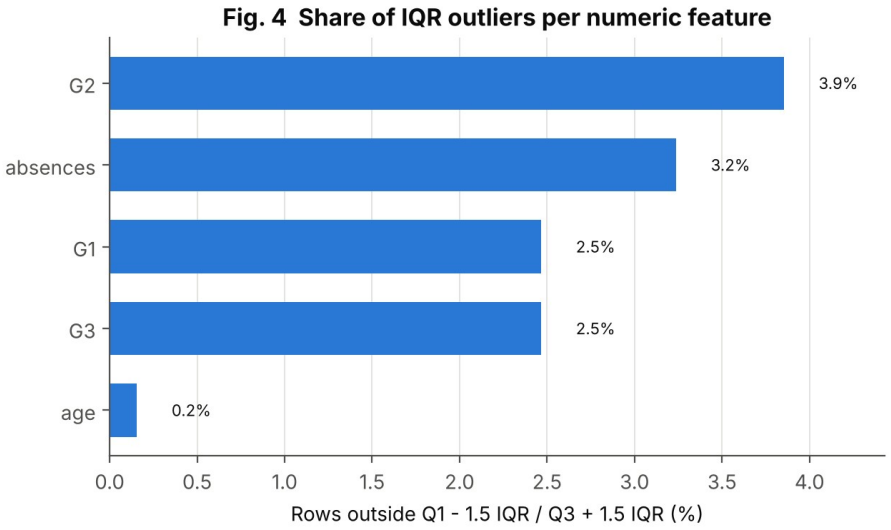


Figure 4: Share of IQR outliers per numeric feature

Aspect	Explanation
Purpose	Quantify how many rows are flagged as outliers.
Variables	Numeric columns with more than five distinct values
Chart type	Sorted horizontal bar chart
Why selected	A ranked bar chart shows which features need a decision.
Insight	Every feature has few flagged rows (maximum 3.9%). The G1/G2/G3 flags are the low-grade and zero-grade students; absences flags very high counts. Because these are genuine values, they were winsorised in the pipeline, not deleted.

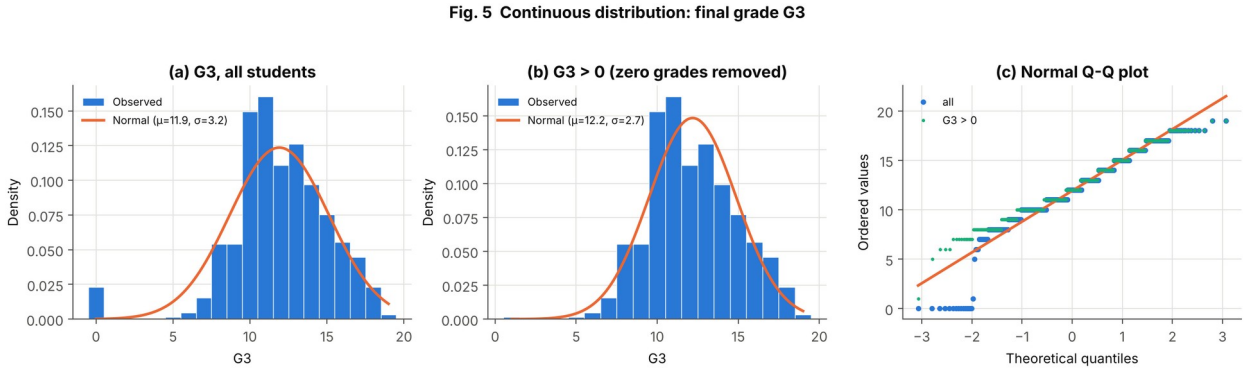


Figure 5: Continuous distribution: final grade G3

Aspect	Explanation
Purpose	Check whether G3 follows a normal distribution, with and without the zero grades.
Variables	G3
Chart type	Histograms with fitted normal curves; normal Q–Q plot
Why selected	The histogram shows the fit visually; the Q–Q plot is more sensitive to the tails.
Insight	All students: skewness -0.91, excess kurtosis 2.71, D'Agostino $p < 0.001$ – clearly not normal because of the long lower tail made by the zeros. Without the 15 zero grades ($n = 634$): mean 12.19, SD 2.69, skewness 0.13, D'Agostino $p = 0.36$, so normality is not rejected. The staircase in the Q–Q plot is caused by G3 taking whole numbers only.

Fig. 6 Discrete distributions: Poisson, negative binomial and Binomial

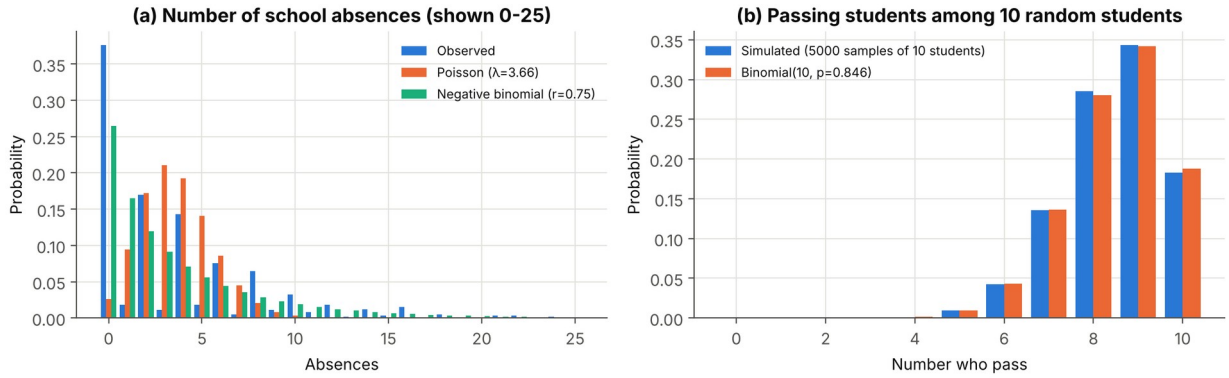


Figure 6: Discrete distributions: Poisson, negative binomial, Binomial

Aspect	Explanation
Purpose	Test count models for absences and illustrate a Binomial model for passing students.
Variables	(a) absences; (b) pass/fail (G3 ≥ 10)
Chart type	Side-by-side bar charts of observed and theoretical probabilities
Why selected	Observed-versus-theoretical bars show directly where a model fits or fails.
Insight	(a) A Poisson with the same mean (λ = 3.66) fits badly: variance/mean = 5.9 (Poisson would be 1) and 37.6% of students have zero absences versus 2.6% expected. A negative binomial (r = 0.75) is closer (total variation distance 0.34 vs 0.50) but still imperfect. (b) Passing is a Bernoulli event with p = 0.846 (95% Wilson interval 0.816–0.872); the number of passes among 10 random students follows Binomial(10, p), confirmed by 5,000 simulated samples. For example, P(all 10 pass) = 0.188.

Fig. 7 Spearman correlation heatmap

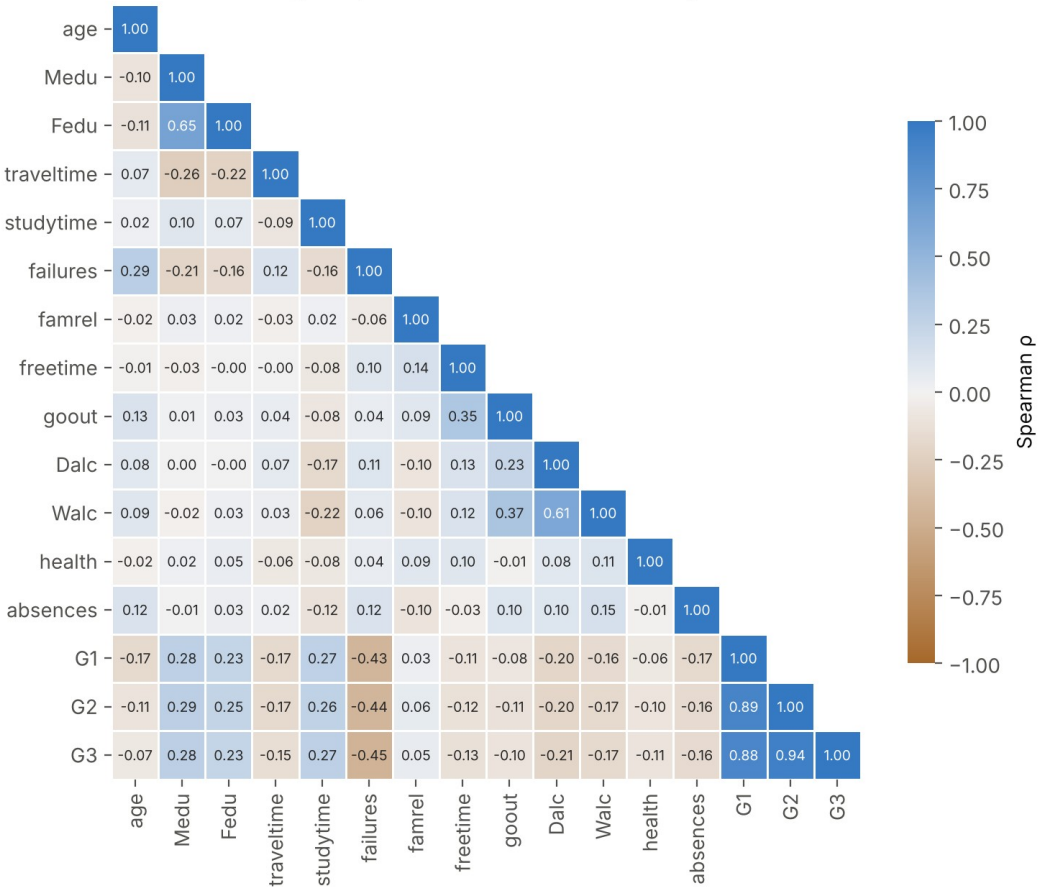


Figure 7: Spearman correlation heatmap

Aspect	Explanation
Purpose	Show monotonic relationships among numeric/ordinal features and G3.
Variables	15 numeric/ordinal features and G3
Chart type	Lower-triangle heat-map with diverging colours and printed coefficients
Why selected	A heat-map is the standard multivariate summary; Spearman is used because many variables are ordinal or skewed.
Insight	The strongest pairs are G2–G3 (ρ = 0.944), G1–G2 (0.893) and G1–G3 (0.883). Mother's and father's education correlate at 0.647, workday and weekend alcohol at 0.613. Among non-grade variables, failures has the strongest link with G3 (ρ = -0.448).

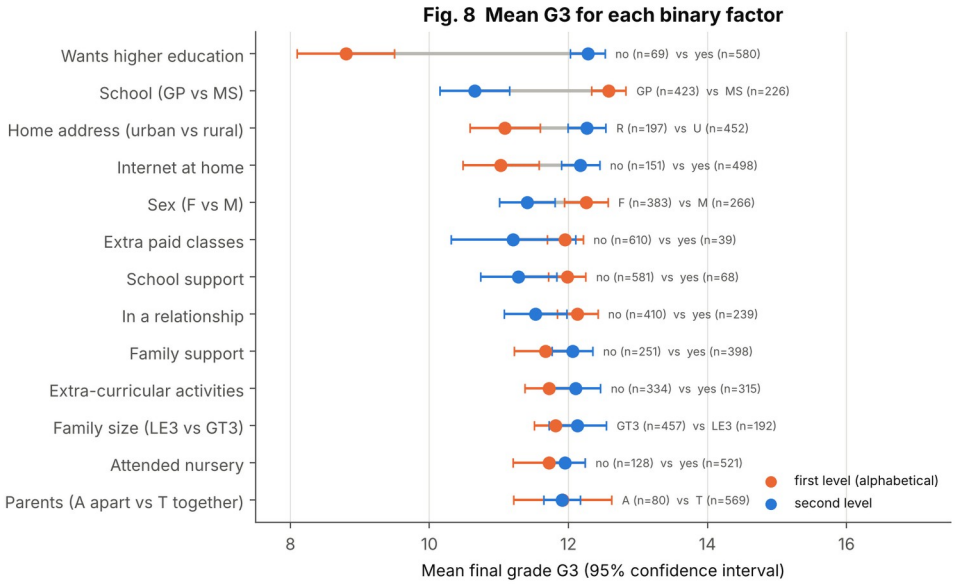


Figure 8: Mean final grade for each binary factor

Aspect	Explanation
Purpose	Compare the average grade between the two levels of every binary factor, with uncertainty.
Variables	13 binary factors vs G3
Chart type	Dot-and-interval chart (mean with 95% CI), sorted by gap
Why selected	A dot-and-interval chart shows the size of each gap and whether the intervals overlap; it needs less ink than 13 pairs of bars.
Insight	The widest gaps are for wanting higher education (8.8 vs 12.3) and school (GP 12.6 vs MS 10.7), followed by address (rural 11.1 vs urban 12.3) and internet at home (11.0 vs 12.2). Intervals for family size, nursery and parents' status overlap strongly. Several factors are unbalanced (for example only 69 students do not want higher education), so wide intervals are expected. These are associations, not causes.

Fig. 9 Earlier grades against the final grade

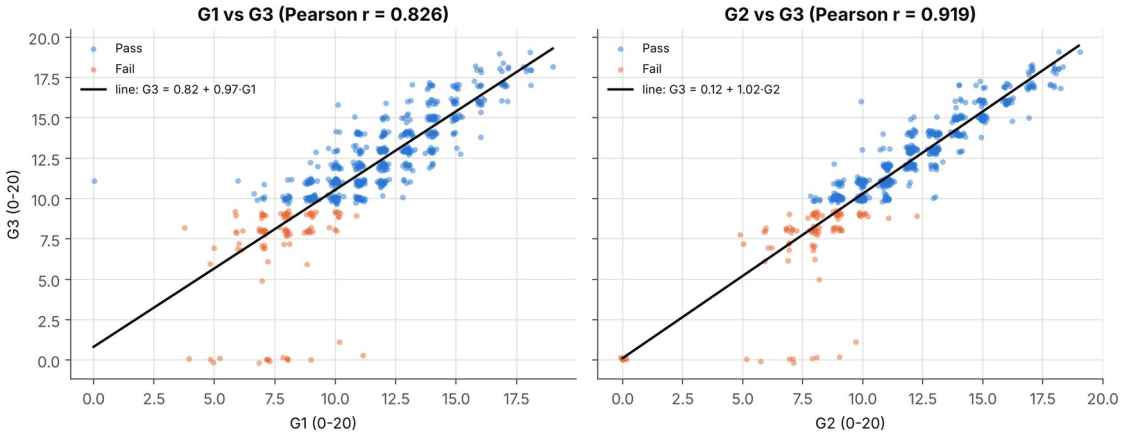


Figure 9: Earlier grades against the final grade

Aspect	Explanation
Purpose	Show how closely G1 and G2 predict G3.
Variables	G1, G2, G3, pass/fail
Chart type	Scatter plots with regression line (jittered, semi-transparent)
Why selected	Scatter plots are the standard bivariate view of two numeric variables.
Insight	Both relationships are close to a straight line (r = 0.826 for G1 and 0.919 for G2). The points far below the line, at G3 = 0, are the zero-grade students. This explains why the model in setting A can be accurate and why it relies on G2 so heavily.

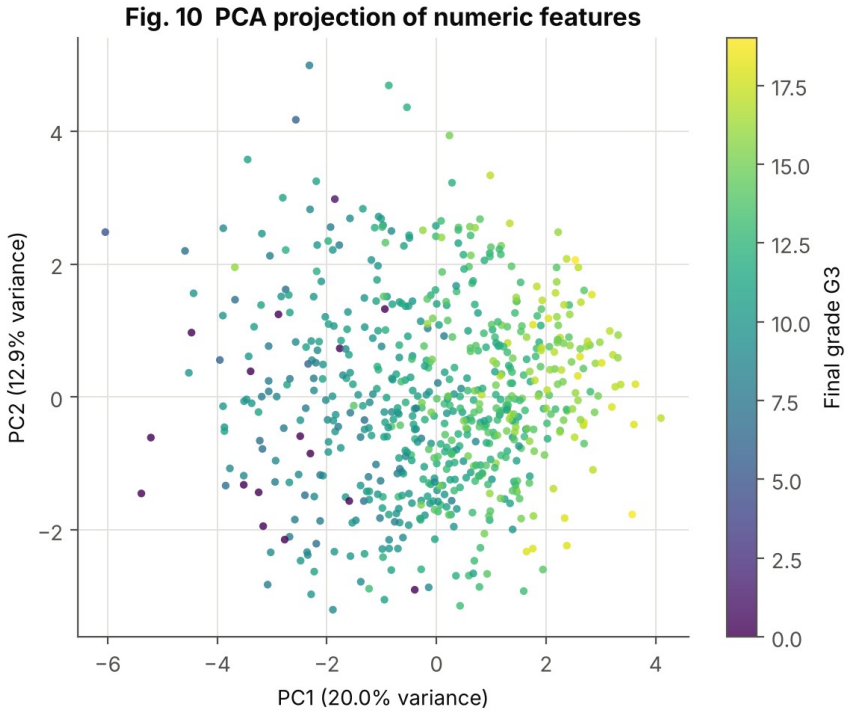


Figure 10: PCA projection of numeric features

Aspect	Explanation
Purpose	See whether grade differences appear when 15 features are compressed into two.
Variables	15 numeric/ordinal features (standardised), coloured by G3
Chart type	PCA scatter with a sequential colour scale
Why selected	PCA is the usual way to view many dimensions in 2-D; a sequential scale suits a numeric colour variable.
Insight	PC1 explains 20.0% and PC2 12.9% of the variance; five components explain 57.9%. G3 rises along PC1 (correlation 0.75 between PC1 and G3), because PC1 is driven mainly by G1, G2 and past failures, but there is no clean cluster structure.

Fig. 11 Pairplot of key variables

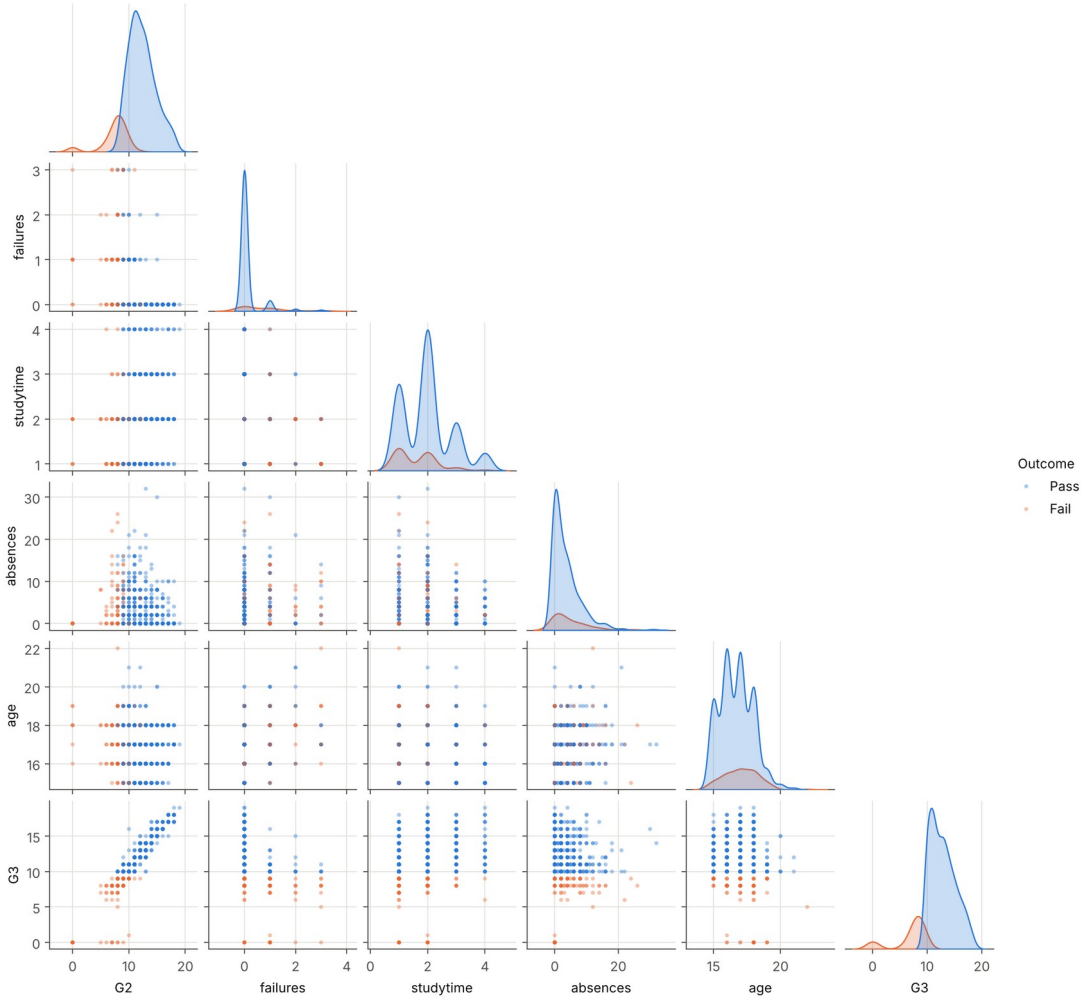


Figure 11: Pairplot of key variables

Aspect	Explanation
Purpose	Show pairwise relations and distributions for six variables.
Variables	G2, failures, studytime, absences, age, G3, pass/fail
Chart type	Pairplot (scatter matrix with KDE diagonals)
Why selected	A pairplot is the quickest way to compare many pairs and class-wise distributions.
Insight	Only G2 separates pass from fail clearly. Compared with passing students, failing students are slightly older (mean age 17.1 vs 16.7), have more past failures (0.75 vs 0.13), study a little less (study-time level 1.6 vs 2.0) and have more absences (4.6 vs 3.5), but the groups overlap heavily on every variable except G2 – a first sign that predictions without G1/G2 will be hard.

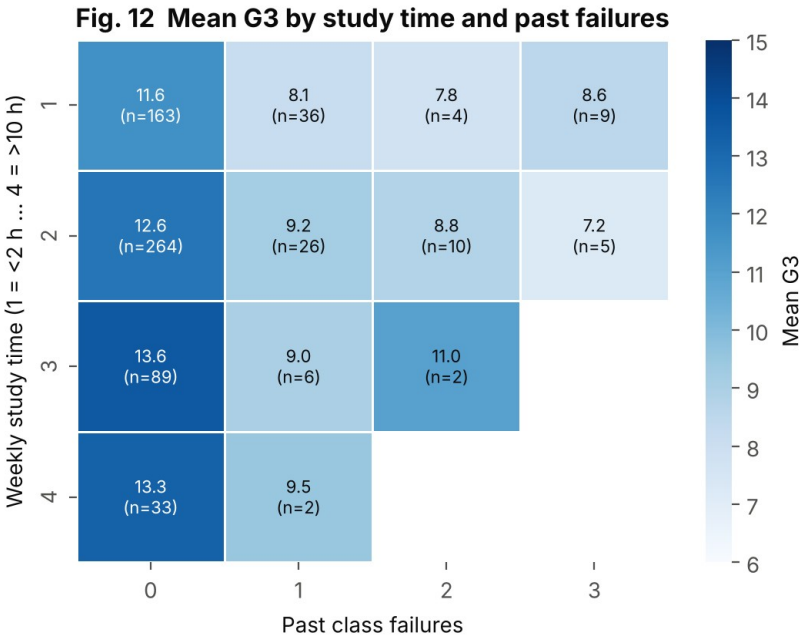


Figure 12: Mean G3 by study time and past failures

Aspect	Explanation
Purpose	Show the joint effect of study time and past failures.
Variables	studytime, failures, G3
Chart type	Annotated heat-map with sequential single-hue scale
Why selected	A two-way table of means shows interaction effects; printing n guards against over-reading small cells.
Insight	Within every row (study time), students with a past failure score about 3–5 points lower than those with none. Within the no-failure column, mean G3 rises from 11.6 (study time 1) to 13.6 (study time 3). Cells with one to six students (for example study time 4 with failures) should not be trusted.

5. Statistical and Distribution-Based Analysis

5.1 Univariate Analysis

- **Centre and spread:** G3 has mean 11.91, median 12, mode 11, SD 3.23 and quartiles 10 and 14 (Table 4).
- **Skewness:** G3 is left-skewed (-0.91) because of the zero grades; absences are strongly right-skewed (2.02); failures are also right-skewed (3.09).
- **Quantiles:** half of the students have two absences or fewer and three quarters have six or fewer, but the maximum is 32.
- **Categorical fields:** 89.4% want higher education, 76.7% have internet at home, 65.2% attend school GP. Some binary fields are very unbalanced (for example only 39 students take extra paid classes).

5.2 Probability Distributions

Table 10: Distribution analysis results

Variable	Model	Result	Conclusion
G3, all students (continuous)	Normal	$\mu = 11.91$, $\sigma = 3.23$; skew -0.91; D'Agostino $p < 0.001$	Rejected: long lower tail from zero grades

Variable	Model	Result	Conclusion
G3 > 0 (continuous)	Normal	n = 634; μ = 12.19, σ = 2.69; skew 0.13; D'Agostino p = 0.36	Not rejected: approximately normal
Pass / fail (discrete)	Bernoulli / Binomial(10, p)	p = 0.846; Wilson 95% CI 0.816–0.872	Binomial matches simulated samples (Fig. 6b)
Absences (count)	Poisson	mean 3.66, variance 21.54, ratio 5.89; zeros 37.6%	Rejected: strongly over-dispersed
Absences (count)	Negative binomial	r = 0.75, p = 0.170 (method of moments); TV distance 0.34	Better than Poisson, still imperfect
Alcohol (Dalc, 1–5)	Frequency distribution	level 1: 69.5%, 2: 18.6%, 3: 6.6%, 4: 2.6%, 5: 2.6%	Geometric-like decline; most students drink little on workdays

In simple words: a *normal* curve describes measurements that cluster symmetrically around an average. G3 would look like that except for the zero grades. A *Poisson* distribution describes counts of rare, independent events and has variance equal to its mean; absences have variance about six times their mean because a few students miss very many days, so a negative binomial, which allows extra spread, fits better.

5.3 Bivariate Analysis

Every input feature was tested against G3. Numeric and ordinal features use **Spearman's ρ**; binary features use the **Mann–Whitney U test** with Cohen's d (standardised mean difference) as effect size; nominal features use the **Kruskal–Wallis H test** with ε^2. Effect sizes are on different scales, so use them to rank features within a test type; the p-value says whether an association is likely to be real, not how important it is.

Table 11: Fourteen features most strongly associated with G3

Feature	Type	Test	Statistic	p-value	Effect size
higher	binary	Mann-Whitney U	6856	< 0.001	1.141
G2	numeric/ordinal	Spearman	0.944	< 0.001	0.944
G1	numeric/ordinal	Spearman	0.883	< 0.001	0.883
school	binary	Mann-Whitney U	64220	< 0.001	0.621
failures	numeric/ordinal	Spearman	-0.448	< 0.001	0.448
address	binary	Mann-Whitney U	35097	< 0.001	0.369
internet	binary	Mann-Whitney U	29582	< 0.001	0.359
Medu	numeric/ordinal	Spearman	0.284	< 0.001	0.284
studytime	numeric/ordinal	Spearman	0.275	< 0.001	0.275
sex	binary	Mann-Whitney U	58916	< 0.001	0.264
Fedu	numeric/ordinal	Spearman	0.235	< 0.001	0.235
paid	binary	Mann-Whitney U	13522	0.149	0.231
schoolsup	binary	Mann-Whitney U	22988	0.026	0.217
Dalc	numeric/ordinal	Spearman	-0.208	< 0.001	0.208

Interpretation. Apart from G1 and G2, the largest effects are for wanting higher education (d = 1.14), school (d = 0.62) and past failures (|ρ| = 0.45). Family background (parents' education, |ρ| about 0.28 and 0.23), study time (0.27) and address (d = 0.37) have smaller effects. Extra paid classes (p = 0.149) show no reliable difference. Nominal fields such as Mjob and Fjob have small ε^2 values. All of these are associations; for example, students who plan to continue studying may also study harder.

5.4 Multivariate Analysis

- **Correlation structure (Fig. 7):** G1, G2 and G3 form one tight block; parents' education and the two alcohol ratings form two weaker pairs; everything else is only loosely related.
- **Multicollinearity:** variance inflation factors are G2 (4.2), G1 (4.2), Walc (1.9), Medu (1.8); none exceeds 5, so multicollinearity is mild even though G1 and G2 are strongly correlated.
- **PCA (Fig. 10):** five components explain 57.9% of variance, so the numeric information is spread over many directions; there is no single dominant lifestyle factor.
- **Combined effects (Fig. 12):** failures and study time act together: the best group (no failures, study time 3) has a mean G3 of 13.6, the worst well-populated group (one failure, study time 1) 8.1.

5.5 Summary of Statistical Findings

Table 12: Statistical findings in plain words

Finding	Evidence
Earlier grades almost determine the final grade	Pearson r = 0.83 (G1) and 0.92 (G2); Fig. 9
Past failures lower the final grade	Mean G3 12.5 (none) vs 8.6 (one or more); ρ = -0.45
Aspiration and school matter	Higher-education wish: 8.8 vs 12.3; school GP 12.6 vs MS 10.7
Lifestyle ratings are weak signals	Most $\|\rho\| < 0.25$ with G3; several effects are small
Absences are not Poisson	Variance/mean = 5.9; 37.6% zeros
Zero grades are a separate group	15 students, all with 0 absences; they break normality of G3

6. Deep Learning Model

6.1 Why a Multilayer Perceptron?

The data are in table form, so the natural deep-learning model is a **multilayer perceptron (MLP)**, a stack of fully connected layers (Goodfellow et al., 2016). CNNs are designed for images and RNN/LSTM/Transformer models for sequences, which this dataset does not contain. Because G3 is a number, the network ends in one linear output neuron and is trained to minimise squared error (regression). To judge whether the extra complexity is worthwhile, a mean-only baseline, ridge regression (Hoerl & Kennard, 1970) and a random forest (Breiman, 2001) were trained on the same data.

6.2 Inputs, Target and Pipeline

Table 13: Model inputs and output

Item	Description
Input, setting A	45 columns from 32 raw features (including G1 and G2)
Input, setting B	43 columns from 30 raw features (no G1, G2)
Target	G3, final grade 0–20 (numeric)
Task type	Regression
Pipeline	Winsorise (1–99 %) → standardise; one-hot for nominal; 0/1 for binary. Fitted on the training set only; saved as models/preprocessor_A.joblib and preprocessor_B.joblib

6.3 Architecture and Hyperparameters

Diagram 2 MLP architecture, setting A (5,441 parameters)

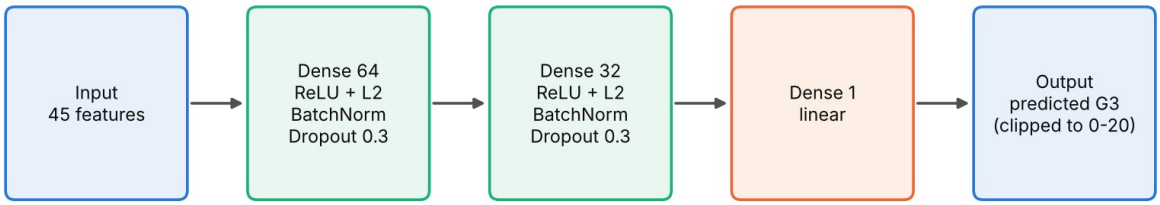


Diagram 2: MLP architecture of the final model (setting A)

Table 14: Final model configuration

Component	Setting A	Setting B	Simple explanation
Hidden layers	64 → 32	32 → 16	Each layer learns combinations of the inputs
Activation	ReLU; linear output	ReLU; linear output	ReLU adds non-linearity; a linear output can give any grade
Regularisation	Dropout 0.3, L2 1e-4, BatchNorm	Dropout 0.3, L2 1e-4, BatchNorm	Reduce over-fitting on only 454 training rows
Loss	Mean squared error	Mean squared error	Penalises large errors strongly
Optimizer	Adam, lr 0.003	Adam, lr 0.003	Adaptive gradient method (Kingma & Ba, 2015)
Batch size / max epochs	32 / 300	32 / 300	Samples per weight update / upper limit
Early stopping	patience 25; ran 81 epochs, best 56	patience 25; ran 45 epochs, best 20	Stops when validation loss stops improving; best weights restored
Parameters	5,441	2,145	Number of learned weights
Random seed	42	42	Makes the run reproducible

6.4 Training, Validation and Testing Procedure

1. Fit the preprocessing pipeline on the training rows and transform all three subsets.
2. For each hyperparameter configuration, train on the training set and monitor loss on the validation set; stop early when it stops improving.
3. Choose the configuration with the lowest **validation RMSE**, separately for settings A and B.
4. Evaluate the chosen models **once** on the untouched test set; report MAE, RMSE and R^2, and analyse the errors.
5. Repeat the whole fit on 10 random splits (hyperparameters fixed) to measure stability.

6.5 Basic Hyperparameter Tuning

A grid search over **3 architectures × 2 dropout rates × 2 learning rates = 12 configurations** was run for each setting; everything else was fixed. Selection used the validation set only. The five best configurations of each setting are shown; the full results are in outputs/model_evaluation.

Table 15: Best five configurations, setting A (sorted by validation RMSE)

Hidden layers	Dropout	Learning rate	Epochs	Val RMSE	Val MAE	Val R^2
(64, 32)	0.3	0.003	81	1.778	1.052	0.781
(32, 16)	0.1	0.003	104	1.805	1.201	0.775
(64, 32)	0.1	0.001	130	1.896	1.132	0.751
(64, 32)	0.1	0.003	67	1.911	1.171	0.747
(32, 16)	0.1	0.001	143	1.931	1.262	0.742

Table 16: Best five configurations, setting B (sorted by validation RMSE)

Hidden layers	Dropout	Learning rate	Epochs	Val RMSE	Val MAE	Val R^2
(32, 16)	0.3	0.003	45	3.608	2.472	0.099
(64, 32, 16)	0.3	0.003	36	3.638	2.530	0.085
(64, 32, 16)	0.1	0.003	29	3.641	2.557	0.083
(32, 16)	0.3	0.001	136	3.648	2.504	0.079
(64, 32, 16)	0.3	0.001	32	3.655	2.517	0.076

Setting A: best configuration [64, 32], dropout 0.3, learning rate 0.003 (validation RMSE 1.778); the spread across the 12 configurations is 0.35 RMSE. Setting B: best [32, 16], dropout 0.3, learning rate 0.003 (validation RMSE 3.608), spread 0.28. With a 97-row validation set, differences of 0.05–0.1 between neighbouring configurations are within noise. Figure 19 shows all results.

6.6 Feature Analysis and Feature Importance

Two complementary methods were used. (1) **Statistical association** (Section 5.3) shows which features relate to G3 in the raw data. (2) **Permutation importance** shows what the *trained model* relies on: each raw feature in the test set is shuffled 10 times and the increase in test RMSE is recorded; a large increase means the model depends on that feature. Shuffling is done on raw columns before encoding, so all one-hot columns of a field are shuffled together.

Table 17: Top eight features by permutation importance, setting A (test set)

Rank	Feature	Mean RMSE increase	Std (10 repeats)
1	G2	1.5039	0.1531
2	G1	0.6255	0.0925
3	goout	0.0788	0.0433
4	absences	0.0715	0.0518
5	reason	0.0652	0.0212
6	romantic	0.0503	0.0249
7	Dalc	0.0450	0.0130
8	activities	0.0384	0.0248

Table 18: Top eight features by permutation importance, setting B (test set)

Rank	Feature	Mean RMSE increase	Std (10 repeats)
1	failures	0.0777	0.0209
2	age	0.0370	0.0326
3	Walc	0.0314	0.0247
4	absences	0.0277	0.0094

Rank	Feature	Mean RMSE increase	Std (10 repeats)
5	studytime	0.0263	0.0226
6	famrel	0.0259	0.0312
7	internet	0.0197	0.0401
8	Dalc	0.0185	0.0254

In setting A the model depends almost entirely on G2 (RMSE rises by 1.50 grade points when G2 is shuffled) and G1 (0.63); every other feature adds less than 0.1. In setting B the largest increase is only 0.078 (past failures), and for most features the error bar is about as large as the bar itself, so the ranking in setting B is uncertain. Importance is shared between correlated features, so a small value does not prove a feature is unrelated to the grade.

7. Model Evaluation and Result Visualization

7.1 Evaluation Metrics

Three regression metrics are used. **MAE** is the average size of the error in grade points. **RMSE** is similar but penalises large errors more. **R^2** is the share of the variation in G3 that the model explains (1 is perfect, 0 is no better than always predicting the average). All values were measured by running the code in the appendix (seed 42). The test set has 98 students that were never used for fitting, scaling or tuning. Predictions are clipped to the valid range 0–20.

Table 19: Regression metrics for setting A (with G1 and G2)

Model	MAE	RMSE	R^2
MLP (test)	0.91	1.33	0.82
MLP (validation)	1.05	1.78	0.78
Ridge regression (test)	0.90	1.35	0.81
Random forest (test)	0.76	1.12	0.87
Mean baseline (test)	2.41	3.13	-0.00

Table 20: Regression metrics for setting B (without G1 and G2)

Model	MAE	RMSE	R^2
MLP (test)	2.03	2.93	0.12
MLP (validation)	2.47	3.61	0.10
Ridge regression (test)	1.87	2.75	0.23
Random forest (test)	1.87	2.73	0.24
Mean baseline (test)	2.41	3.13	-0.00

Reading the tables. With earlier grades the MLP is wrong by about 0.91 grade points on average (RMSE 1.33); 67.3% of test students are predicted within one point and 91.8% within two. Without them, the average error is 2.03 points and R^2 is only 0.12: the MLP is only slightly better than always predicting the average grade (RMSE 3.13). As an extra check, the predicted grade was turned into a pass/fail call (predicted ≥ 9.5 means pass): accuracy 93.9% and AUC 0.965 in setting A, accuracy 79.6% and AUC 0.647 in setting B. Only 15 of the 98 test students fail, so these pass/fail figures are very uncertain; always predicting 'pass' would already score 84.7%.

7.2 Comparison with Baseline Models

On the seed-42 test set the random forest has the lowest error in both settings (RMSE 1.12 and 2.73); ridge regression is about as accurate as the MLP in setting A (RMSE 1.35 vs 1.33) and better in setting B. **The MLP is therefore competitive with, but not better than, the simpler models.** This is common for small tabular data, where the main relationship (G3 depends almost linearly on G2) is easy to capture. The MLP is kept as the required deep-learning model; the baselines give context.

7.3 Result Visualizations and Interpretation

Fig. 13 Training vs validation loss (MSE, standardised target)

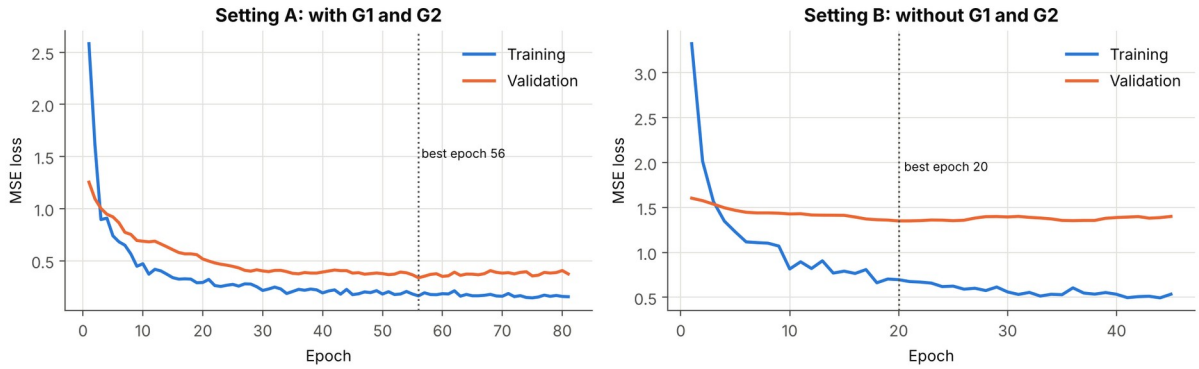


Figure 13: Training vs validation loss

Aspect	Explanation
Purpose	Show how learning progressed and detect over-fitting.
Variables	Epoch, MSE loss (train and validation), settings A and B
Chart type	Line charts with best-epoch marker
Why selected	Loss curves are the standard diagnostic of neural-network training.
Insight	In both settings training loss keeps falling while validation loss flattens, and the gap is larger in setting B. Early stopping restored the weights from epoch 56 (A, of 81 run) and epoch 20 (B, of 45). The loss is on the standardised target, so a validation loss of about 0.35 corresponds roughly to an RMSE of 0.59 × 3.1 ≈ 1.8 grade points.

Fig. 14 Training vs validation MAE (standardised target)

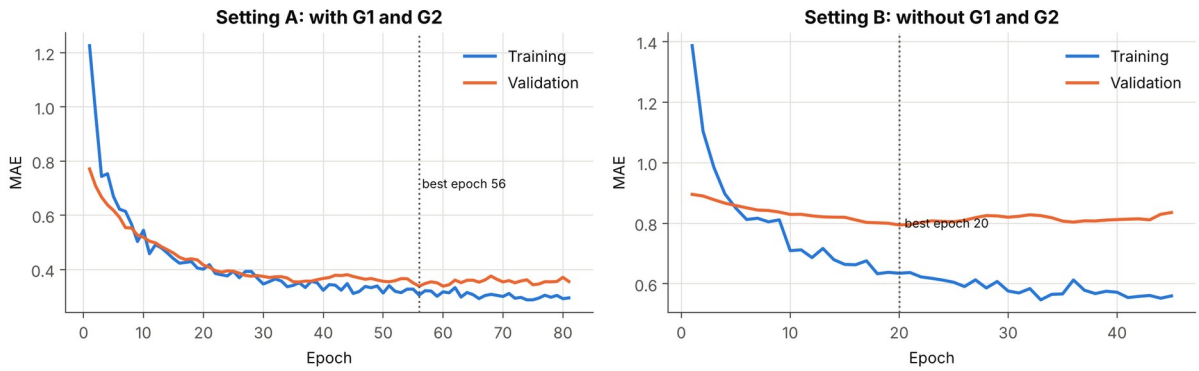


Figure 14: Training vs validation MAE

Aspect	Explanation
Purpose	Show the mean absolute error progress for both subsets.
Variables	Epoch, MAE (train and validation)
Chart type	Line charts
Why selected	MAE is easier to interpret than squared loss.
Insight	The pattern repeats Figure 13: validation MAE levels off long before training MAE does. In setting B the validation MAE stays high and nearly flat, which means the network finds little learnable signal in the background features.

Fig. 15 Predicted vs actual final grade (test set)

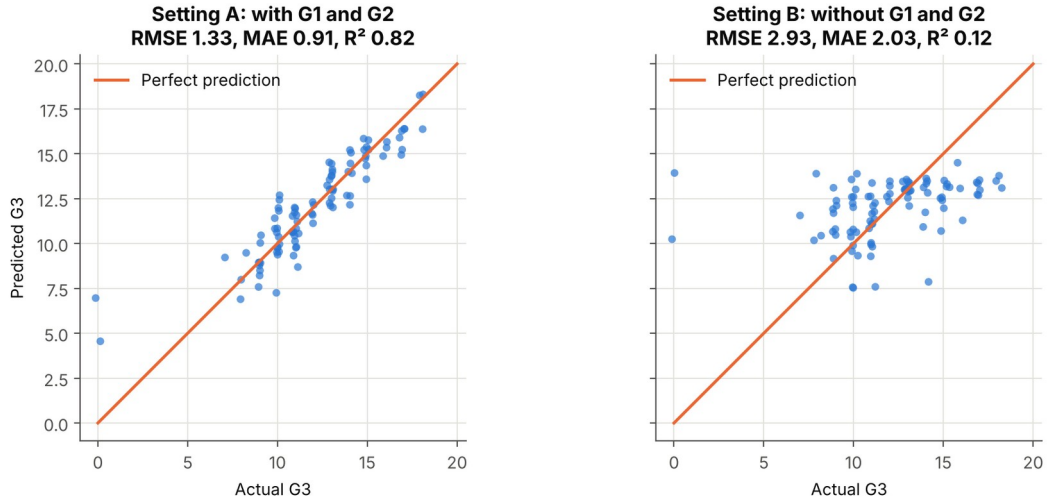


Figure 15: Predicted vs actual final grade

Aspect	Explanation
Purpose	Show how close predictions are to the truth.
Variables	Actual and predicted G3 on the test set
Chart type	Scatter plot with the perfect-prediction line (jittered)
Why selected	For regression, points on the diagonal are perfect predictions; the spread shows the error.
Insight	Setting A: points hug the diagonal, with the largest misses at the low end (actual 0, predicted 7.0). Setting B: predictions are squeezed into a narrow band (about 8–14) whatever the true grade, so high and low achievers are not separated; the zero-grade student is predicted at about 14.

Fig. 16 Error distribution (test set)

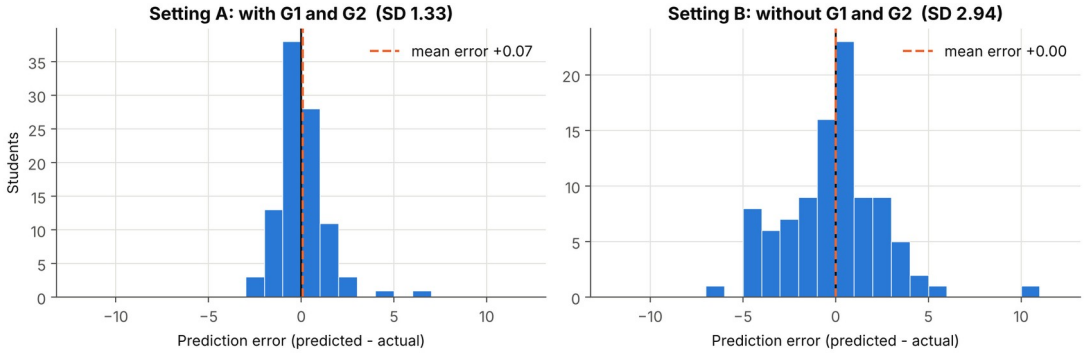


Figure 16: Error distribution

Aspect	Explanation
Purpose	Show how prediction errors are spread.
Variables	Error = predicted − actual (test set)
Chart type	Histograms with zero line and mean error
Why selected	Regression error distributions reveal bias and heavy tails.
Insight	Errors are centred near zero in both settings (mean +0.07 and +0.00), so there is no systematic over- or under-prediction. The spread is much larger without G1/G2 (SD 2.94 vs 1.33). Shapiro–Wilk rejects normal errors ($p < 0.001$ in both) because of a few large misses, mainly zero-grade students.

Fig. 17 Residual plot (test set)

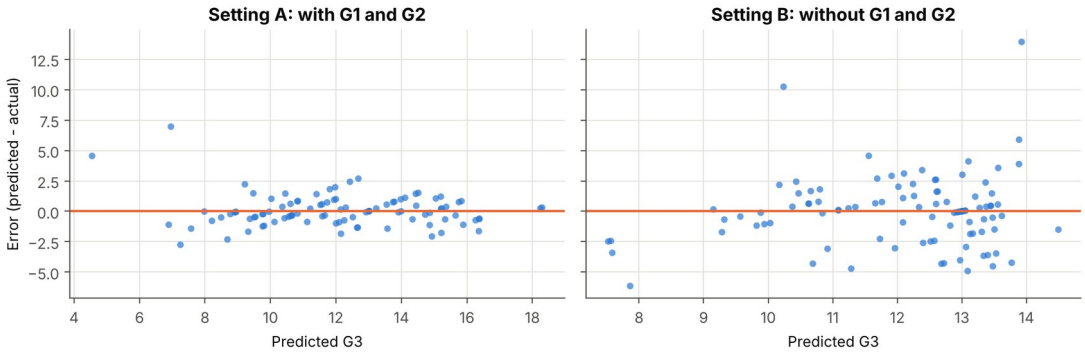


Figure 17: Residual plot

Aspect	Explanation
Purpose	Check whether errors depend on the predicted value.
Variables	Predicted G3, error
Chart type	Scatter plot with zero line
Why selected	A good model shows a shapeless cloud around zero.
Insight	Setting A shows no strong pattern except two isolated large positive errors (zero-grade students predicted at about 4.6 and 7.0; the worst absolute error is 7.0 points). Setting B shows a much wider scatter: predictions stay between about 7.5 and 14.5, errors of ±5 points are common, and the two largest errors (about 10 and 14 points) belong to students whose actual grade was 0. The model cannot tell most students apart.

Fig. 18 Permutation importance (top 10 features)

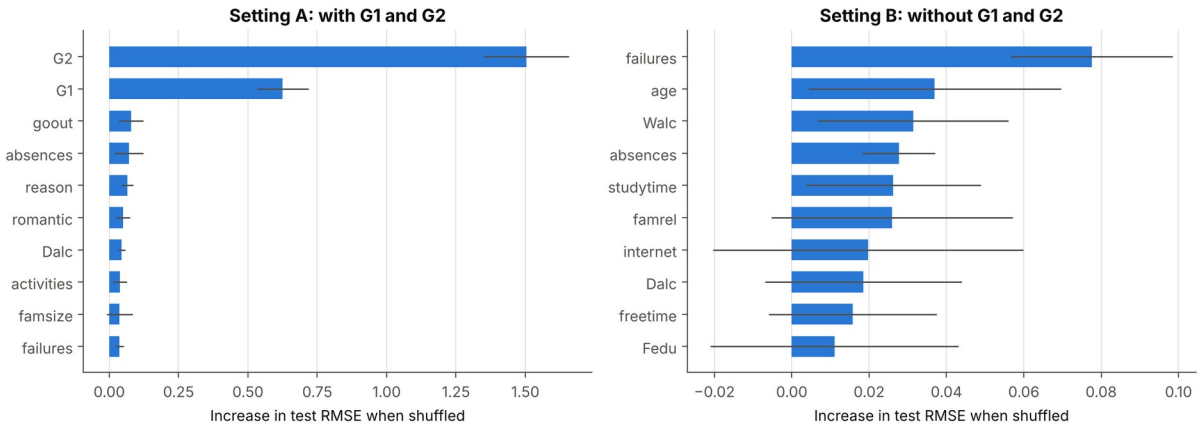


Figure 18: Permutation importance (top 10 features)

Aspect	Explanation
Purpose	Show which raw features each trained model depends on.
Variables	Raw features; increase in test RMSE after shuffling
Chart type	Horizontal bars with error bars (SD of 10 shuffles)
Why selected	Model-agnostic and works with one-hot encoded inputs when applied to raw columns.
Insight	Setting A is dominated by G2 and G1 (note the different x-axis scales). In setting B the bars are about 20 times smaller and error bars overlap zero for several features, so only past failures stands out; the others should not be over-interpreted.

Fig. 19 Hyperparameter search results

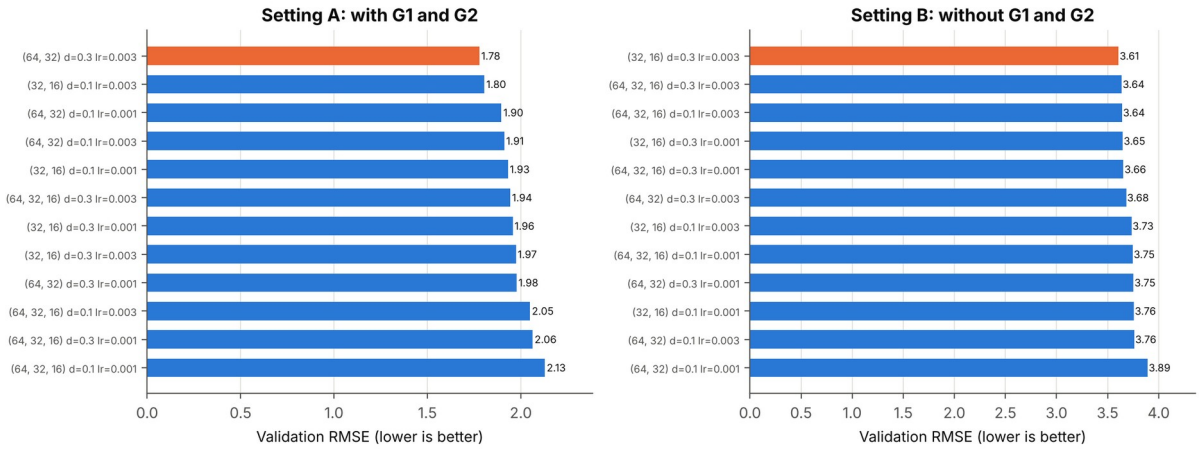


Figure 19: Hyperparameter search results

Aspect	Explanation
Purpose	Compare all 12 configurations per setting.
Variables	Architecture, dropout, learning rate vs validation RMSE
Chart type	Sorted horizontal bars; best highlighted
Why selected	A ranked chart shows both the winner and how close the others are.
Insight	Setting A validation RMSE ranges from 1.78 to 2.13; setting B from 3.61 to 3.89. Neighbouring configurations are close, so the choice of the winner is not very certain.

7.4 Stability over 10 Random Splits

Because one 98-student test set is small, the complete procedure (preprocessing, training, evaluation) was repeated for 10 different random splits, keeping the hyperparameters chosen above. Table 21 gives the mean and standard deviation of test errors; Figure 20 shows the spread.

Table 21: Test errors over 10 random splits (mean ± standard deviation)

Setting	Model	MAE	RMSE	R^2
A	Mean baseline	2.35 ± 0.14	3.14 ± 0.22	-0.00 ± 0.00
A	Ridge regression	0.84 ± 0.10	1.30 ± 0.19	0.83 ± 0.04
A	Random forest	0.80 ± 0.06	1.22 ± 0.18	0.84 ± 0.05
A	MLP	0.93 ± 0.09	1.40 ± 0.19	0.80 ± 0.05
B	Mean baseline	2.35 ± 0.14	3.14 ± 0.22	-0.00 ± 0.00
B	Ridge regression	2.00 ± 0.20	2.80 ± 0.29	0.20 ± 0.11
B	Random forest	2.01 ± 0.15	2.75 ± 0.32	0.23 ± 0.13
B	MLP	2.12 ± 0.16	2.90 ± 0.28	0.14 ± 0.13

Fig. 20 Stability over 10 random train/validation/test splits

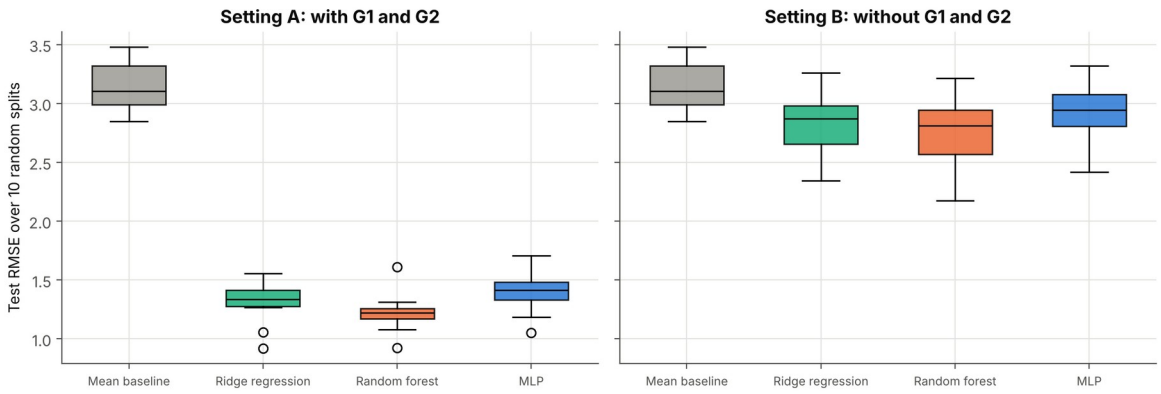


Figure 20: Stability over 10 random train/validation/test splits

Interpretation. In setting A the ranking is random forest (RMSE 1.22), ridge (1.30), MLP (1.40); the differences are comparable to the split-to-split standard deviation (about 0.2), so they are small. In setting B all three models are only modestly better than the mean baseline (3.14): MLP 2.90, ridge 2.80, random forest 2.75, with R^2 of only 0.14–0.23. Note that the hyperparameters were chosen using the seed-42 validation set, which may overlap the test sets of other splits, so these repeated-split numbers are slightly optimistic for the MLP.

8. Conclusion, Challenges and Future Scope

8.1 What Was Learned from the Data

- The dataset is clean in the sense of having no missing values or duplicates, but it contains a small group of zero grades (15 students) that behave differently from the rest.
- The final grade is almost a continuation of the earlier grades: r = 0.83 with G1 and 0.92 with G2.
- Among other factors, past failures (mean G3 12.5 vs 8.6), wish for higher education (12.3 vs 8.8) and school (12.6 vs 10.7) are most strongly associated with G3.
- Lifestyle ratings (free time, going out, alcohol, health) show weak relationships, and absences follow an over-dispersed count distribution that a Poisson model cannot describe.

8.2 Model Performance

With G1 and G2, the tuned MLP reached test MAE 0.91, RMSE 1.33 and R^2 0.82. Without them it reached MAE 2.03, RMSE 2.93 and R^2 0.12. The MLP did not outperform ridge regression or the random forest, and over 10 splits the three models were close to each other. The main lesson is that how much information is available (the earlier grades) matters far more than which algorithm is used.

8.3 Challenges and Limitations

- **Small data:** 454 training rows make a neural network easy to over-fit; dropout, L2, BatchNorm and early stopping were needed, and results vary by about ±0.2 RMSE between splits.
- **Dependence on G1 and G2:** setting A is accurate but is available only after two periods of the course; setting B is the real early-warning case and performs poorly.
- **Zero grades:** a few students with G3 = 0 produce the largest errors and affect the normality of G3 and of the residuals. Their meaning is not documented in the dataset description.
- **Association, not causation:** group differences (for example higher-education wish, school) cannot be interpreted as causes.
- **Hyperparameter choice on one validation set:** the repeated-split results reuse those hyperparameters, so they slightly favour the MLP.
- **Single course and region:** the data cover one Portuguese-language course in two schools; results may not transfer elsewhere.
- **Data source for this study:** the CSV was obtained from public copies of the UCI dataset (identical across two copies) because the UCI site was not reachable from the analysis environment.
- **Ethics:** sex, family and alcohol information are sensitive; a deployed system must be checked for fairness and used to offer support, not to label students.

8.4 Future Scope

- Model the zero-grade students separately (for example a two-stage model: first predict whether the final assessment is taken, then predict the grade).
- Use repeated cross-validation or a nested procedure for tuning, with confidence intervals for model comparisons.
- Combine this file with the Mathematics file using a student-grouped split, to get more data without leakage.
- Try gradient-boosted trees, ordinal regression and a classification of grade bands, and calibrate predictions with intervals.
- Add SHAP explanations for individual students.
- Build a teacher dashboard that shows predicted risk, with fairness checks across groups.

9. Literature Review and References

9.1 Short Literature Review

Dataset and application. Cortez and Silva (2008) collected school reports and questionnaires from two Portuguese secondary schools and applied data-mining models to predict student performance in Mathematics and Portuguese language, studying settings with and without earlier-period grades. The data were later released on the UCI repository (Cortez, 2008) under a CC BY 4.0 licence. This project reuses the Portuguese-language file and follows the same idea of comparing predictions with and without G1 and G2, using a neural network and simple baselines.

EDA and visualisation. Tukey (1977) established exploratory analysis. Fry (2007) formalised the seven-stage visualisation process (acquire, parse, filter, mine, represent, refine, interact) followed in this report. Plots and statistics were made with Matplotlib (Hunter, 2007), seaborn (Waskom, 2021) and SciPy (Virtanen et al., 2020).

Modelling. Goodfellow, Bengio and Courville (2016) describe multilayer perceptrons and regularisation; Srivastava et al. (2014) introduced dropout and Kingma and Ba (2015) the Adam optimiser. The baselines use scikit-learn (Pedregosa et al., 2011), ridge regression (Hoerl & Kennard, 1970) and random forests (Breiman, 2001). The network was built with TensorFlow/Keras (Abadi et al., 2016).

9.2 References (APA 7th edition)

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., et al. (2016). TensorFlow: A system for large-scale machine learning. *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*, 265–283.

Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32.

Cortez, P. (2008). *Student performance* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5TG7T

Cortez, P., & Silva, A. (2008). Using data mining to predict secondary school student performance. In A. Brito & J. Teixeira (Eds.), *Proceedings of 5th FUture BUsiness TEChnology Conference (FUBUTEC 2008)* (pp. 5–12). EUROSIS.

Fry, B. (2007). *Visualizing data: Exploring and explaining data with the Processing environment*. O'Reilly Media.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, *12*(1), 55–67.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *3rd International Conference on Learning Representations (ICLR 2015)*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, *15*(56), 1929–1958.

Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.

Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*, 261–272.

Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, *6*(60), 3021.

10. Rubric Alignment

The table maps each assessment criterion in the assignment sheet to the evidence in this report and repository. It lists where the evidence is; it does not claim any marks.

Table 22: Rubric alignment

CO / Criterion (marks)	Report section	Evidence provided
CO3 – Selection and creation of visualizations (3)	4.1, 4.4, Figs. 1–12; Ch. 7, Figs. 13–20	20 figures of 12+ types (bar, histogram, boxplot, scatter, heat-map, pairplot, Q–Q, PCA, dot-and-interval, line, residual, importance) chosen by data type; 2 interactive pages

CO / Criterion (marks)	Report section	Evidence provided
CO3 – Quality and presentation (2)	4.2	Numbered titles, labelled axes, legends, direct labels, colour-blind-safe palette, consistent colours
CO3 – Interpretation of visualizations (3)	4.4, 7.3	Purpose / variables / chart type / why / insight table for every figure
CO4 – Univariate analysis and distribution exploration (4)	2.5, 5.1, Figs. 1–5	Mean, median, mode, SD, quartiles, skew, histograms, boxplots, outlier analysis
CO4 – Discrete / continuous distributions (4)	5.2, Figs. 5–6	Normal fit and Q–Q with and without zeros; Bernoulli/Binomial; Poisson and negative binomial for absences
CO4 – Bivariate / multivariate analysis (4)	5.3, 5.4, Figs. 7–12	Spearman, Mann–Whitney, Kruskal–Wallis with effect sizes, heat-maps, VIF, PCA, pairplot, scatter
CO4 – Interpretation of patterns and findings (4)	5.5, 3.7	Plain-language findings table linked to evidence
CO5 – Real-world problem / application (3)	1.2–1.5, 2.2	Early prediction of final grades from school and family data
CO5 – Application of EDA techniques (3)	Chapter 3	Missing/duplicate checks, outliers, encoding, scaling, stratified split, leakage control, two settings
CO5 – Insights and interpretation (3)	3.7, 5.5, 4.4	Zero-grade finding, failures and aspiration effects, over-dispersed absences, dependence on G1/G2
CO5 – Conclusion and recommendations (3)	Chapter 8	Findings, model performance, limitations, future scope
CO5 – GitHub repository and documentation (4)	Appendix B	Repository structure, README, requirements, dataset, code, report; public link to be added after upload
Enrichment – ML/DL (optional)	Ch. 6, 7	Tuned MLP regressor in two settings, baselines, permutation importance, error analysis, 10-split stability

Appendix A – Complete Python Source Code

Files are shown in the order in which the pipeline runs. Run everything with: python src/run_all.py

A.1 config.py – paths, seed, column groups, plot style

```
"""Project-wide configuration: paths, random seed, column groups and plot style."""
from pathlib import Path
import matplotlib
matplotlib.use("Agg")                      # headless backend (no display needed)
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]
DATA_PATH = ROOT / "data" / "student-por.csv"
EDA_DIR = ROOT / "outputs" / "eda"
EVAL_DIR = ROOT / "outputs" / "model_evaluation"
MODEL_DIR = ROOT / "models"
for d in (EDA_DIR, EVAL_DIR, MODEL_DIR):
    d.mkdir(parents=True, exist_ok=True)

SEED = 42
TARGET = "G3"                              # final grade, 0-20 (regression target)
PASS_MARK = 10                             # G3 >= 10 counts as "pass" in the Portuguese system
GRADE_COLS = ["G1", "G2"]                  # earlier period grades

# ---- Column groups (32 input features) -------------------------------------
BINARY_COLS = ["school", "sex", "address", "famsize", "Pstatus", "schoolsup", "famsup", "paid",
               "activities", "nursery", "higher", "internet", "romantic"]
NOMINAL_COLS = ["Mjob", "Fjob", "reason", "guardian"]
# Everything else is numeric: ordinal 1-5 ratings and counts are kept as numbers.
ORDINAL_COLS = ["Medu", "Fedu", "traveltime", "studytime", "failures", "famrel", "freetime",
                "goout", "Dalc", "Walc", "health"]

# ---- Plot style: validated categorical slots 1-3 + neutral inks --------------
BLUE, ORANGE, GREEN = "#2a78d6", "#eb6834", "#1baf7a"
INK, INK2, GRID = "#0b0b0b", "#52514e", "#e4e3df"
PASS_COLORS = {"Pass": BLUE, "Fail": ORANGE}

def set_style():
    plt.rcParams.update({
        "figure.dpi": 110, "savefig.dpi": 200, "savefig.bbox": "tight",
        "font.size": 10, "axes.titlesize": 12, "axes.titleweight": "bold",
        "axes.labelsize": 10, "axes.edgecolor": INK2, "axes.labelcolor": INK,
        "text.color": INK, "xtick.color": INK2, "ytick.color": INK2,
        "axes.spines.top": False, "axes.spines.right": False,
        "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.7,
        "axes.axisbelow": True, "legend.frameon": False,
        "figure.facecolor": "white", "axes.facecolor": "white",
    })

def save_fig(fig, path):
    fig.savefig(path)
    plt.close(fig)
```

A.2 data_preprocessing.py – loading, cleaning, split, pipeline

```
"""Data loading, cleaning checks, splitting and the leakage-safe preprocessing pipeline."""
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler
from config import (DATA_PATH, TARGET, SEED, BINARY_COLS, NOMINAL_COLS, GRADE_COLS)


def load_data(path=DATA_PATH) -> pd.DataFrame:
    """Load the UCI 'Student Performance' (Portuguese course) CSV (comma separated)."""
    df = pd.read_csv(path)
    df.columns = df.columns.str.strip()
    return df


def numeric_cols(df: pd.DataFrame):
    """Numeric features = everything that is neither a binary flag nor a nominal code (target excluded)."""
    return [c for c in df.columns if c not in BINARY_COLS + NOMINAL_COLS + [TARGET]]


def clean_data(df: pd.DataFrame):
    """Remove exact duplicate rows; report missing values. Returns (clean_df, report dict)."""
    report = {"rows_before": len(df), "missing_cells": int(df.isna().sum().sum()),
              "duplicate_rows": int(df.duplicated().sum())}
    df = df.drop_duplicates().reset_index(drop=True)
    report["rows_after"] = len(df)
    return df, report


class Winsorizer(BaseEstimator, TransformerMixin):
    """Clip each column to its [lower, upper] percentile, learned on TRAINING data only."""
    def __init__(self, lower=1, upper=99):
```

```
        self.lower, self.upper = lower, upper

    def fit(self, X, y=None):
        X = np.asarray(X, dtype=float)
        self.lo_ = np.percentile(X, self.lower, axis=0)
        self.hi_ = np.percentile(X, self.upper, axis=0)
        return self

    def transform(self, X):
        return np.clip(np.asarray(X, dtype=float), self.lo_, self.hi_)


def feature_columns(df, use_grades=True):
    """Input columns for the two experimental settings:
       use_grades=True  -> all 32 features (includes G1, G2)
       use_grades=False -> 30 features (early-warning setting: no earlier grades)."""
    cols = [c for c in df.columns if c != TARGET]
    return cols if use_grades else [c for c in cols if c not in GRADE_COLS]


def build_preprocessor(columns) -> ColumnTransformer:
    """numeric -> winsorise -> standardise | nominal -> one-hot | binary -> 0/1 ordinal encoding."""
    num = [c for c in columns if c not in BINARY_COLS + NOMINAL_COLS]
    return ColumnTransformer([
        ("num", Pipeline([("clip", Winsorizer(1, 99)), ("scale", StandardScaler())]), num),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), [c for c in NOMINAL_COLS if c in
columns]),
        ("bin", OrdinalEncoder(), [c for c in BINARY_COLS if c in columns]),
    ])


def split_data(df: pd.DataFrame, seed=SEED, val_size=0.15, test_size=0.15):
    """70/15/15 split. Stratified on grade band (fail <10, 10-13, >=14) so every subset has a similar grade mix."""
    X, y = df.drop(columns=[TARGET]), df[TARGET].astype(float)
    band = pd.cut(y, [-1, 9, 13, 20], labels=False)
    X_tr, X_tmp, y_tr, y_tmp, b_tr, b_tmp = train_test_split(
        X, y, band, test_size=val_size + test_size, stratify=band, random_state=seed)
    X_val, X_te, y_val, y_te = train_test_split(
        X_tmp, y_tmp, test_size=test_size / (val_size + test_size), stratify=b_tmp, random_state=seed)
    return X_tr, X_val, X_te, y_tr, y_val, y_te
```

A.3 eda.py – statistics and distribution fitting

```
"""Statistical EDA: inspection, outliers, univariate / bivariate / multivariate statistics and
distribution fitting. Writes CSV tables and results_eda.json (used by the report)."""
import json
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from config import EDA_DIR, TARGET, BINARY_COLS, NOMINAL_COLS, PASS_MARK
from data_preprocessing import load_data, clean_data, numeric_cols


def inspect(df):
    """Dtypes + descriptive statistics (mean/median/mode/std/quantiles/skew/kurtosis) of numeric columns."""
    num = numeric_cols(df) + [TARGET]
    pd.DataFrame({"dtype": df.dtypes.astype(str), "n_unique": df.nunique(),
                  "missing": df.isna().sum()}).to_csv(EDA_DIR / "table_dtypes.csv")
    rows = []
    for c in num:
        s = df[c]
        rows.append({"feature": c, "mean": s.mean(), "median": s.median(), "mode": s.mode().iloc[0],
                     "std": s.std(), "min": s.min(), "q25": s.quantile(.25), "q75": s.quantile(.75),
                     "max": s.max(), "skew": s.skew(), "kurtosis": s.kurt()})
    desc = pd.DataFrame(rows).set_index("feature"); desc.round(3).to_csv(EDA_DIR / "table_descriptive_numeric.csv")
    return desc


def outlier_table(df):
    """IQR (1.5x) and |z|>3 outlier counts for numeric columns with more than 5 distinct values."""
    rows = []
    for c in numeric_cols(df) + [TARGET]:
        s = df[c]
        if s.nunique() <= 5:
            continue
        q1, q3 = s.quantile([.25, .75]); iqr = q3 - q1; lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
        n_iqr = int(((s < lo) | (s > hi)).sum())
        rows.append({"feature": c, "lower_fence": lo, "upper_fence": hi, "iqr_outliers": n_iqr,
                     "iqr_pct": 100 * n_iqr / len(s), "z_gt_3": int((np.abs((s - s.mean()) / s.std()) > 3).sum())})
    t = pd.DataFrame(rows).set_index("feature").sort_values("iqr_pct", ascending=False)
    t.round(3).to_csv(EDA_DIR / "table_outliers.csv"); return t


def bivariate_tests(df):
    """Association of every input feature with the numeric target G3.
    numeric/ordinal -> Spearman rho (+p); binary -> Mann-Whitney U with Cohen's d (mean diff / pooled SD);
    nominal (>2 levels) -> Kruskal-Wallis H with epsilon^2 effect size."""
    n = len(df); rows = []
    for c in numeric_cols(df):
        r, p = stats.spearmanr(df[c], df[TARGET])
        rows.append({"feature": c, "type": "numeric/ordinal", "test": "Spearman", "statistic": r, "p_value": p,
"effect_size": abs(r)})
    for c in BINARY_COLS:
```

```
        g = [x[TARGET].values for _, x in df.groupby(c)]
        U, p = stats.mannwhitneyu(g[0], g[1]); sp = np.sqrt(((len(g[0]) - 1) * g[0].var(ddof=1) + (len(g[1]) - 1) *
g[1].var(ddof=1)) / (n - 2))
        d = (g[1].mean() - g[0].mean()) / sp
        rows.append({"feature": c, "type": "binary", "test": "Mann-Whitney U", "statistic": U, "p_value": p,
"effect_size": abs(d)})
    for c in NOMINAL_COLS:
        g = [x[TARGET].values for _, x in df.groupby(c)]; H, p = stats.kruskal(*g)
        rows.append({"feature": c, "type": "nominal", "test": "Kruskal-Wallis", "statistic": H, "p_value": p,
"effect_size": (H - len(g) + 1) / (n - len(g))})
    t = pd.DataFrame(rows).set_index("feature").sort_values("effect_size", ascending=False)
    t.to_csv(EDA_DIR / "table_bivariate_tests.csv"); return t


def multivariate(df):
    """Spearman matrix (numeric + target), VIF on numeric features and PCA."""
    num = numeric_cols(df)
    corr = df[num + [TARGET]].corr(method="spearman"); corr.round(3).to_csv(EDA_DIR / "table_spearman_corr.csv")
    X = StandardScaler().fit_transform(df[num]); vif = {}
    for i, c in enumerate(num):
        o = np.delete(X, i, 1); r2 = LinearRegression().fit(o, X[:, i]).score(o, X[:, i]); vif[c] = 1 / (1 - r2)
    vif = pd.Series(vif, name="VIF").sort_values(ascending=False); vif.round(2).to_csv(EDA_DIR / "table_vif.csv")
    return corr, vif, PCA().fit(X).explained_variance_ratio_


def distribution_fits(df):
    """Continuous: normal fit of G3 (with and without the zero grades). Discrete: Poisson vs negative binomial
    for absences; Bernoulli/Binomial for pass/fail; frequency distribution of alcohol ratings."""
    out = {}; g3 = df[TARGET]; nz = g3[g3 > 0]
    mu, sd = stats.norm.fit(g3); mu2, sd2 = stats.norm.fit(nz)
    out["G3_all"] = {"mean": mu, "std": sd, "skew": g3.skew(), "excess_kurtosis": g3.kurt(),
                     "dagostino_p": stats.normaltest(g3)[1], "ks_D": stats.kstest(g3, stats.norm(mu,
sd).cdf).statistic}
    out["G3_nonzero"] = {"n": int(len(nz)), "mean": mu2, "std": sd2, "skew": nz.skew(), "excess_kurtosis": nz.kurt(),
                         "dagostino_p": stats.normaltest(nz)[1], "ks_D": stats.kstest(nz, stats.norm(mu2,
sd2).cdf).statistic}
    out["n_G3_zero"] = int((g3 == 0).sum())
    ab = df["absences"]; m, v = ab.mean(), ab.var()
    out["absences"] = {"mean": m, "variance": v, "dispersion_index": v / m, "zero_share": float((ab == 0).mean())}
    # negative binomial by method of moments (r, p): mean = r(1-p)/p, var = r(1-p)/p^2
    p_nb = m / v; r_nb = m * p_nb / (1 - p_nb); out["absences"].update(nb_r=r_nb, nb_p=p_nb)
    k = np.arange(0, int(ab.max()) + 1); obs = ab.value_counts(normalize=True).reindex(k, fill_value=0).values
    out["absences"]["tv_poisson"] = 0.5 * float(np.abs(obs - stats.poisson.pmf(k, m)).sum())
    out["absences"]["tv_negbin"] = 0.5 * float(np.abs(obs - stats.nbinom.pmf(k, r_nb, p_nb)).sum())
    ps = (g3 >= PASS_MARK).mean(); lo, hi = stats.binomtest(int((g3 >= PASS_MARK).sum()),
len(g3)).proportion_ci(method="wilson")
    out["pass_rate"] = ps; out["pass_wilson_ci"] = [lo, hi]
    for c in ["Dalc", "Walc"]:
        out[c + "_pct"] = {int(k): round(100 * v, 1) for k, v in
df[c].value_counts(normalize=True).sort_index().items()}
    return out


def run():
    raw = load_data(); df, rep = clean_data(raw)
    desc = inspect(df); ot = outlier_table(df); bt = bivariate_tests(df); corr, vif, evr = multivariate(df)
    out = {"shape": list(raw.shape), "clean_report": rep, "n_numeric": len(numeric_cols(df)), "n_binary":
len(BINARY_COLS),
           "n_nominal": len(NOMINAL_COLS), "dtypes": pd.Series(df.dtypes.astype(str)).value_counts().to_dict(),
           "g3_desc": desc.loc[TARGET].round(3).to_dict(), "pass_count": int((df[TARGET] >= PASS_MARK).sum()),
           "outliers": ot.round(2).reset_index().to_dict("records"), "bivariate":
bt.reset_index().round(5).to_dict("records"),
           "vif": vif.round(2).to_dict(), "pca_var_ratio": [round(float(x), 4) for x in evr],
           "distribution_fits": distribution_fits(df)}
    # group means used in the narrative
    gm = {}
    for c in ["school", "higher", "sex", "failures", "studytime", "schoolsup", "internet", "address", "romantic",
"paid"]:
        gm[c] = df.groupby(c)[TARGET].agg(["mean", "size"]).round(2).reset_index().astype(object).values.tolist()
    out["group_means"] = gm
    out["g3_zero_profile"] = {"mean_G2_of_zero": float(df[df[TARGET] == 0]["G2"].mean()), "mean_absences_of_zero":
float(df[df[TARGET] == 0]["absences"].mean()),
                              "mean_absences_others": float(df[df[TARGET] > 0]["absences"].mean()),
                              "zero_with_G2_zero": int(((df[TARGET] == 0) & (df["G2"] == 0)).sum())}
    c = corr.where(np.triu(np.ones(corr.shape, bool), 1)).stack().sort_values(key=abs, ascending=False)
    out["top_corr_pairs"] = [[a, b, round(float(v), 3)] for (a, b), v in c.head(8).items()]
    out["pearson_G2_G3"] = float(df["G2"].corr(df[TARGET])); out["pearson_G1_G3"] = float(df["G1"].corr(df[TARGET]))
    with open(EDA_DIR / "results_eda.json", "w") as fh:
        json.dump(out, fh, indent=1, default=lambda o: o.item() if hasattr(o, "item") else str(o))
    print(json.dumps({k: out[k] for k in ["shape", "clean_report", "pass_count", "g3_zero_profile", "top_corr_pairs",
"pearson_G2_G3"]}, indent=1, default=str))
    print(bt.head(14)[["type", "statistic", "p_value", "effect_size"]].round(4)); print(vif.head(5));
print(out["distribution_fits"])
    return df


if __name__ == "__main__":
    run()
```

A.4 visualizations.py – Figures 1–12 and interactive pages

```
"""EDA visualisations (Figures 1-12) + two interactive Plotly pages (stage 7: Interact)."""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from scipy import stats
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from config import (EDA_DIR, TARGET, PASS_MARK, BLUE, ORANGE, GREEN, PASS_COLORS, SEED,
                    BINARY_COLS, set_style, save_fig)
from data_preprocessing import load_data, clean_data, numeric_cols

NICE = {"higher": "Wants higher education", "school": "School (GP vs MS)", "address": "Home address (urban vs rural)",
        "internet": "Internet at home", "sex": "Sex (F vs M)", "paid": "Extra paid classes", "schoolsup": "School
support",
        "romantic": "In a relationship", "famsup": "Family support", "activities": "Extra-curricular activities",
        "nursery": "Attended nursery", "famsize": "Family size (LE3 vs GT3)", "Pstatus": "Parents (A apart vs T
together)"}


def _pf(df):
    return np.where(df[TARGET] >= PASS_MARK, "Pass", "Fail")


def fig01_target(df):
    """Distribution of the target G3 (histogram with the pass mark)."""
    fig, ax = plt.subplots(figsize=(7, 3.9)); s = df[TARGET]
    ax.bar(*np.unique(s, return_counts=True), color=[ORANGE if v < PASS_MARK else BLUE for v in np.unique(s)],
width=0.8)
    ax.axvline(PASS_MARK - 0.5, color=INK2 if False else "#52514e", ls="--", lw=1.3); ax.set_ylim(0, 118);
ax.text(PASS_MARK - 0.3, 112, "pass mark (10)", fontsize=8)
    ax.annotate(f"{(s == 0).sum()} students\nwith G3 = 0", xy=(0, (s == 0).sum()), xytext=(2.2, 70), fontsize=8,
arrowprops=dict(arrowstyle="->", color="#52514e"))
    ax.set(title="Fig. 1  Distribution of the final grade (G3)", xlabel="Final grade G3 (0-20)", ylabel="Number of
students")
    save_fig(fig, EDA_DIR / "fig01_target_distribution.png")


def fig02_hist(df):
    """Histograms with mean (solid) and median (dashed) for six numeric variables."""
    cols = ["G1", "G2", "age", "absences", "studytime", "failures"]
    fig, axes = plt.subplots(2, 3, figsize=(11, 6))
    for ax, c in zip(axes.ravel(), cols):
        s = df[c]; bins = np.arange(s.min() - .5, s.max() + 1.5, 1) if s.nunique() < 25 else 30
        ax.hist(s, bins=bins, color=BLUE, edgecolor="white", linewidth=0.5)
        ax.axvline(s.mean(), color="#0b0b0b", lw=1.6, label=f"mean {s.mean():.1f}"); ax.axvline(s.median(),
color=ORANGE, lw=1.6, ls="--", label=f"median {s.median():.1f}")
        ax.set(title=c, xlabel=c, ylabel="Frequency"); ax.legend(fontsize=8)
    fig.suptitle("Fig. 2  Histograms of key numeric variables", fontweight="bold", y=1.0); fig.tight_layout()
    save_fig(fig, EDA_DIR / "fig02_histograms.png")


def fig03_box(df):
    """Boxplots of G3 grouped by five factors."""
    cols = ["school", "failures", "studytime", "higher", "Dalc"]
    labs = {"school": "School", "failures": "Past failures", "studytime": "Weekly study time (1-4)", "higher": "Wants
higher education", "Dalc": "Workday alcohol (1-5)"}
    fig, axes = plt.subplots(1, 5, figsize=(14, 3.8), sharey=True)
    for ax, c in zip(axes, cols):
        sns.boxplot(data=df, x=c, y=TARGET, color=BLUE, width=0.6, linewidth=1, fliersize=2, ax=ax)
        ax.set_title(labs[c], fontsize=9); ax.set_xlabel(""); ax.set_ylabel("Final grade G3" if c == "school" else "")
    fig.suptitle("Fig. 3  Final grade (G3) by key factors", fontweight="bold", y=1.03); fig.tight_layout()
    save_fig(fig, EDA_DIR / "fig03_boxplots_g3_by_factor.png")


def fig04_outliers(df):
    """Share of IQR outliers per numeric feature."""
    t = pd.read_csv(EDA_DIR / "table_outliers.csv", index_col=0).sort_values("iqr_pct")
    fig, ax = plt.subplots(figsize=(7, 3.8)); ax.barh(t.index, t["iqr_pct"], color=BLUE, height=0.65)
    for y, v in enumerate(t["iqr_pct"]): ax.text(v + 0.2, y, f"{v:.1f}%", va="center", fontsize=8)
    ax.set(title="Fig. 4  Share of IQR outliers per numeric feature", xlabel="Rows outside Q1 - 1.5 IQR / Q3 + 1.5 IQR
(%)"); ax.grid(axis="y", visible=False)
    ax.set_xlim(0, t["iqr_pct"].max() * 1.15); save_fig(fig, EDA_DIR / "fig04_outlier_share.png")


def fig05_normal(df):
    """Continuous distribution: normal fit and Q-Q plot of G3 for (a) all students, (b) students with G3 > 0."""
    fig, axes = plt.subplots(1, 3, figsize=(13, 3.8)); s = df[TARGET]; nz = s[s > 0]
    for ax, d, ttl in [(axes[0], s, "(a) G3, all students"), (axes[1], nz, "(b) G3 > 0 (zero grades removed)")]:
        mu, sd = stats.norm.fit(d); ax.hist(d, bins=np.arange(d.min() - .5, d.max() + 1.5, 1), density=True,
color=BLUE, edgecolor="white", linewidth=.5, label="Observed")
        x = np.linspace(d.min(), d.max(), 300); ax.plot(x, stats.norm.pdf(x, mu, sd), color=ORANGE, lw=2,
label=f"Normal (μ={mu:.1f}, σ={sd:.1f})")
        ax.set(title=ttl, xlabel="G3", ylabel="Density"); ax.legend(fontsize=8)
    stats.probplot(s, dist="norm", plot=axes[2]); axes[2].get_lines()[0].set(color=BLUE, markersize=3, label="all");
axes[2].get_lines()[1].set(color=ORANGE, lw=2)
    z = stats.probplot(nz, dist="norm", fit=False); axes[2].plot(z[0], z[1], ".", color=GREEN, markersize=3, label="G3
> 0")
    axes[2].set(title="(c) Normal Q-Q plot", xlabel="Theoretical quantiles", ylabel="Ordered values");
axes[2].legend(fontsize=8)
    fig.suptitle("Fig. 5  Continuous distribution: final grade G3", fontweight="bold", y=1.03); fig.tight_layout()
    save_fig(fig, EDA_DIR / "fig05_normal_fit_qq.png")


def fig06_discrete(df):
    """Discrete distributions: (a) Poisson vs negative binomial for absences, (b) Binomial(10, p) for passes."""
    ab = df["absences"]; m = ab.mean(); v = ab.var(); p_nb = m / v; r_nb = m * p_nb / (1 - p_nb)
    k = np.arange(0, 26); obs = ab.value_counts(normalize=True).reindex(k, fill_value=0)
    fig, (a, b) = plt.subplots(1, 2, figsize=(11, 3.9)); w = 0.28
    a.bar(k - w, obs.values, width=w, color=BLUE, label="Observed"); a.bar(k, stats.poisson.pmf(k, m), width=w,
color=ORANGE, label=f"Poisson (λ={m:.2f})")
    a.bar(k + w, stats.nbinom.pmf(k, r_nb, p_nb), width=w, color=GREEN, label=f"Negative binomial (r={r_nb:.2f})")
```

```
    a.set(title="(a) Number of school absences (shown 0-25)", xlabel="Absences", ylabel="Probability");
a.legend(fontsize=8)
    p = (df[TARGET] >= PASS_MARK).mean(); n = 10; rng = np.random.default_rng(SEED)
    sims = np.array([(df[TARGET].sample(n, random_state=int(r)) >= PASS_MARK).sum() for r in rng.integers(0, 10**6,
5000)])
    kk = np.arange(0, n + 1); emp = np.bincount(sims, minlength=n + 1) / len(sims)
    b.bar(kk - .2, emp, width=.4, color=BLUE, label="Simulated (5000 samples of 10 students)"); b.bar(kk + .2,
stats.binom.pmf(kk, n, p), width=.4, color=ORANGE, label=f"Binomial(10, p={p:.3f})")
    b.set(title="(b) Passing students among 10 random students", xlabel="Number who pass", ylabel="Probability");
b.legend(fontsize=8, loc="upper left")
    fig.suptitle("Fig. 6  Discrete distributions: Poisson, negative binomial and Binomial", fontweight="bold", y=1.03);
fig.tight_layout()
    save_fig(fig, EDA_DIR / "fig06_discrete_distributions.png")


def fig07_corr(df):
    """Spearman correlation heat-map (numeric/ordinal features and G3)."""
    cols = numeric_cols(df) + [TARGET]; corr = df[cols].corr(method="spearman"); fig, ax = plt.subplots(figsize=(8.4,
7))
    sns.heatmap(corr, mask=np.triu(np.ones_like(corr, bool), 1), cmap=sns.diverging_palette(40, 250, s=85, l=50,
as_cmap=True), center=0, vmin=-1, vmax=1,
                annot=True, fmt=".2f", annot_kws={"size": 7}, linewidths=.4, linecolor="white", cbar_kws={"label":
"Spearman ρ", "shrink": .7}, ax=ax)
    ax.grid(False); ax.set_title("Fig. 7  Spearman correlation heatmap"); save_fig(fig, EDA_DIR /
"fig07_correlation_heatmap.png")


def fig08_group_means(df):
    """Mean G3 with 95% CI for the two levels of each binary factor, sorted by size of the gap."""
    rows = []
    for c in BINARY_COLS:
        for lv, g in df.groupby(c)[TARGET]:
            rows.append({"factor": c, "level": lv, "mean": g.mean(), "ci": 1.96 * g.std(ddof=1) / np.sqrt(len(g)), "n":
len(g)})
    t = pd.DataFrame(rows); gap = t.groupby("factor")["mean"].agg(lambda s: abs(s.iloc[0] - s.iloc[1])).sort_values()
    fig, ax = plt.subplots(figsize=(7.6, 5.4)); y = np.arange(len(gap))
    for i, f in enumerate(gap.index):
        a, b = t[t.factor == f].iloc[0], t[t.factor == f].iloc[1]
        ax.plot([a["mean"], b["mean"]], [i, i], color="#b9b8b2", lw=2, zorder=1)
        ax.errorbar(a["mean"], i, xerr=a["ci"], fmt="o", color=ORANGE, ms=7, capsize=3, zorder=3);
ax.errorbar(b["mean"], i, xerr=b["ci"], fmt="o", color=BLUE, ms=7, capsize=3, zorder=3)
        ax.text(max(a["mean"] + a["ci"], b["mean"] + b["ci"]) + .15, i, f"{a['level']} (n={a['n']})  vs  {b['level']}
(n={b['n']})", va="center", fontsize=7, color="#52514e")
    ax.set_yticks(y); ax.set_yticklabels([NICE.get(f, f) for f in gap.index]); ax.set(xlabel="Mean final grade G3 (95%
confidence interval)", xlim=(7.5, 17.5), title="Fig. 8  Mean G3 for each binary factor")
    ax.scatter([], [], color=ORANGE, label="first level (alphabetical)"); ax.scatter([], [], color=BLUE, label="second
level"); ax.legend(loc="lower right", fontsize=8); ax.grid(axis="y", visible=False)
    save_fig(fig, EDA_DIR / "fig08_group_means.png")


def fig09_scatter(df):
    """Scatter plots of G1 and G2 against G3 with regression line (jittered, coloured by pass/fail)."""
    rng = np.random.default_rng(SEED); pf = _pf(df); fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.6), sharey=True)
    for ax, c in zip(axes, ["G1", "G2"]):
        for lab in ["Pass", "Fail"]:
            m = pf == lab; ax.scatter(df[c][m] + rng.normal(0, .12, m.sum()), df[TARGET][m] + rng.normal(0, .12,
m.sum()), s=14, alpha=.5, color=PASS_COLORS[lab], edgecolor="none", label=lab)
        b, a = np.polyfit(df[c], df[TARGET], 1); xs = np.array([df[c].min(), df[c].max()]); ax.plot(xs, a + b * xs,
color="#0b0b0b", lw=1.6, label=f"line: G3 = {a:.2f} + {b:.2f}·{c}")
        r = df[c].corr(df[TARGET]); ax.set(title=f"{c} vs G3 (Pearson r = {r:.3f})", xlabel=f"{c} (0-20)", ylabel="G3
(0-20)" if c == "G1" else ""); ax.legend(fontsize=8, loc="upper left")
    fig.suptitle("Fig. 9  Earlier grades against the final grade", fontweight="bold", y=1.02); fig.tight_layout();
save_fig(fig, EDA_DIR / "fig09_grade_scatter.png")


def fig10_pca(df):
    """PCA projection of standardised numeric features, coloured by G3 (sequential scale)."""
    num = numeric_cols(df); Z = StandardScaler().fit_transform(df[num]); pca = PCA(2, random_state=SEED).fit(Z); P =
pca.transform(Z)
    fig, ax = plt.subplots(figsize=(6.6, 5.2)); sc = ax.scatter(P[:, 0], P[:, 1], c=df[TARGET], cmap="viridis", s=14,
alpha=.8, edgecolor="none")
    cb = fig.colorbar(sc, ax=ax); cb.set_label("Final grade G3"); ev = pca.explained_variance_ratio_ * 100
    ax.set(title="Fig. 10  PCA projection of numeric features", xlabel=f"PC1 ({ev[0]:.1f}% variance)", ylabel=f"PC2
({ev[1]:.1f}% variance)"); save_fig(fig, EDA_DIR / "fig10_pca.png")


def fig11_pair(df):
    """Pairplot of key variables coloured by pass/fail."""
    d = df[["G2", "failures", "studytime", "absences", "age", TARGET]].copy(); d["Outcome"] = _pf(df)
    g = sns.pairplot(d, hue="Outcome", hue_order=["Pass", "Fail"], palette=PASS_COLORS, corner=True, plot_kws={"s": 9,
"alpha": .4, "edgecolor": "none"}, diag_kind="kde", height=1.8)
    g.figure.suptitle("Fig. 11  Pairplot of key variables", fontweight="bold", y=1.01); g.figure.savefig(EDA_DIR /
"fig11_pairplot.png"); plt.close(g.figure)


def fig12_heat(df):
    """Mean G3 for each combination of study time and past failures (cell counts printed)."""
    m = df.pivot_table(index="studytime", columns="failures", values=TARGET, aggfunc="mean"); n =
df.pivot_table(index="studytime", columns="failures", values=TARGET, aggfunc="size")
    fig, ax = plt.subplots(figsize=(6.2, 4.4)); sns.heatmap(m, annot=False, cmap="Blues", vmin=6, vmax=15,
linewidths=1, linecolor="white", cbar_kws={"label": "Mean G3"}, ax=ax)
    for i in range(m.shape[0]):
        for j in range(m.shape[1]):
            if not np.isnan(m.iloc[i, j]): ax.text(j + .5, i + .5, f"{m.iloc[i, j]:.1f}\n(n={int(n.iloc[i, j])})",
ha="center", va="center", fontsize=8, color="white" if m.iloc[i, j] > 11.5 else "#0b0b0b")
    ax.grid(False); ax.set(title="Fig. 12  Mean G3 by study time and past failures", xlabel="Past class failures",
ylabel="Weekly study time (1 = <2 h … 4 = >10 h)"); save_fig(fig, EDA_DIR / "fig12_studytime_failures.png")
```

```
def interactive(df):
    """Stage 7 (Interact): hover / zoom / legend-filter pages built with Plotly."""
    import plotly.express as px
    d = df.copy(); d["Outcome"] = _pf(df)
    f1 = px.scatter(d, x="G2", y=TARGET, color="Outcome", color_discrete_map=PASS_COLORS, opacity=.6, hover_data=["G1",
"failures", "studytime", "absences", "school", "higher"], title="Interactive: second-period grade vs final grade (click
legend to filter)")
    f1.write_html(EDA_DIR / "interactive_g2_vs_g3.html", include_plotlyjs="cdn")
    f2 = px.box(d, x="failures", y=TARGET, color="school", points="all", title="Interactive: final grade by past
failures and school")
    f2.write_html(EDA_DIR / "interactive_g3_by_failures.html", include_plotlyjs="cdn")


def run():
    set_style(); df, _ = clean_data(load_data())
    for f in (fig01_target, fig02_hist, fig03_box, fig04_outliers, fig05_normal, fig06_discrete, fig07_corr,
fig08_group_means, fig09_scatter, fig10_pca, fig11_pair, fig12_heat, interactive):
        f(df); print("done", f.__name__)


if __name__ == "__main__":
    run()
```

A.5 train_model.py – tuning, training, baselines, repeated splits

```
"""Train the regression MLP for two experimental settings and run the baselines.
  Setting A: all 32 features (includes the earlier grades G1 and G2)
  Setting B: 30 features, NO earlier grades (early-warning setting)
Hyperparameters are chosen by grid search on the VALIDATION set only (split seed 42).
Then a repeated-split experiment (10 random splits) measures how stable the results are."""
import itertools, json, os, time
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
import joblib
import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from config import MODEL_DIR, EVAL_DIR, SEED, PASS_MARK
from data_preprocessing import load_data, clean_data, split_data, build_preprocessor, feature_columns

BATCH_SIZE, MAX_EPOCHS, PATIENCE = 32, 300, 25
GRID = {"hidden": [(32, 16), (64, 32), (64, 32, 16)], "dropout": [0.1, 0.3], "lr": [1e-3, 3e-3]}
SETTINGS = {"A": True, "B": False}                    # name -> use_grades


def build_mlp(n_in, hidden, dropout, lr):
    """MLP regressor: [Dense(ReLU, L2) -> BatchNorm -> Dropout] x L -> Dense(1, linear)."""
    layers = [tf.keras.layers.Input((n_in,))]
    for h in hidden:
        layers += [tf.keras.layers.Dense(h, activation="relu", kernel_regularizer=tf.keras.regularizers.l2(1e-4)),
                   tf.keras.layers.BatchNormalization(), tf.keras.layers.Dropout(dropout)]
    layers.append(tf.keras.layers.Dense(1))
    m = tf.keras.Sequential(layers)
    m.compile(optimizer=tf.keras.optimizers.Adam(lr), loss="mse", metrics=["mae"])
    return m


def fit_mlp(Xtr, ytr, Xval, yval, hidden, dropout, lr, seed=SEED):
    """Train with early stopping on validation loss. The target is standardised (train mean/std) for stable
training."""
    tf.keras.utils.set_random_seed(seed)
    mu, sd = ytr.mean(), ytr.std()
    m = build_mlp(Xtr.shape[1], hidden, dropout, lr)
    es = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=PATIENCE, restore_best_weights=True)
    h = m.fit(Xtr, (ytr - mu) / sd, validation_data=(Xval, (yval - mu) / sd), epochs=MAX_EPOCHS,
              batch_size=BATCH_SIZE, callbacks=[es], verbose=0)
    return m, h.history, (mu, sd)


def predict(m, X, scale):
    """Back-transform to grade points and clip to the valid 0-20 range."""
    return np.clip(m.predict(X, verbose=0).ravel() * scale[1] + scale[0], 0, 20)


def reg_metrics(y, p):
    return {"MAE": mean_absolute_error(y, p), "RMSE": float(np.sqrt(mean_squared_error(y, p))), "R2": r2_score(y, p)}


def prepare(df, use_grades, seed=SEED):
    cols = feature_columns(df, use_grades); Xtr, Xval, Xte, ytr, yval, yte = split_data(df, seed)
    pre = build_preprocessor(cols).fit(Xtr[cols])                    # fitted on TRAIN only (no leakage)
    f = lambda X: pre.transform(X[cols]).astype("float32")
    return pre, f(Xtr), f(Xval), f(Xte), ytr.values, yval.values, yte.values


def run():
    df, _ = clean_data(load_data()); summary = {"batch_size": BATCH_SIZE, "max_epochs": MAX_EPOCHS, "patience":
PATIENCE, "settings": {}}
    for name, use_g in SETTINGS.items():
        pre, Xtr, Xval, Xte, ytr, yval, yte = prepare(df, use_g); joblib.dump(pre, MODEL_DIR /
f"preprocessor_{name}.joblib")
        rows, best = [], None
```

```
        for hidden, dr, lr in itertools.product(GRID["hidden"], GRID["dropout"], GRID["lr"]):
            t0 = time.time(); m, h, sc = fit_mlp(Xtr, ytr, Xval, yval, hidden, dr, lr)
            vm = reg_metrics(yval, predict(m, Xval, sc))
            rec = {"hidden": str(hidden), "dropout": dr, "lr": lr, "epochs_run": len(h["loss"]), "val_RMSE":
vm["RMSE"], "val_MAE": vm["MAE"], "val_R2": vm["R2"], "seconds": round(time.time() - t0, 1)}
            rows.append(rec); print(name, rec)
            if best is None or vm["RMSE"] < best["rmse"]:
                best = {"rmse": vm["RMSE"], "model": m, "hist": h, "scale": sc, "cfg": {"hidden": list(hidden),
"dropout": dr, "lr": lr}}
        pd.DataFrame(rows).to_csv(EVAL_DIR / f"tuning_results_{name}.csv", index=False)
        best["model"].save(MODEL_DIR / f"mlp_setting_{name}.keras")
        json.dump({k: [float(x) for x in v] for k, v in best["hist"].items()}, open(EVAL_DIR / f"history_{name}.json",
"w"))
        summary["settings"][name] = {"use_grades": use_g, "n_inputs": int(Xtr.shape[1]), "best_config": best["cfg"],
"scale": [float(best["scale"][0]), float(best["scale"][1])],
                                     "epochs_run": len(best["hist"]["loss"]), "val_RMSE": best["rmse"], "n_params":
int(best["model"].count_params()),
                                     "sizes": {"train": len(ytr), "val": len(yval), "test": len(yte)}}
    # ---- repeated random splits: stability of every model (config fixed from the seed-42 search) -------
    reps = []
    for seed in range(10):
        for name, use_g in SETTINGS.items():
            pre, Xtr, Xval, Xte, ytr, yval, yte = prepare(df, use_g, seed); c = summary["settings"][name]
["best_config"]
            m, h, sc = fit_mlp(Xtr, ytr, Xval, yval, tuple(c["hidden"]), c["dropout"], c["lr"], seed=seed)
            models = {"MLP": predict(m, Xte, sc)}
            for nm, mdl in {"Mean baseline": DummyRegressor(), "Ridge regression": Ridge(alpha=10.0), "Random forest":
RandomForestRegressor(300, random_state=seed, n_jobs=-1, min_samples_leaf=2)}.items():
                mdl.fit(Xtr, ytr); models[nm] = np.clip(mdl.predict(Xte), 0, 20)
            for nm, p in models.items():
                reps.append({"seed": seed, "setting": name, "model": nm, **reg_metrics(yte, p)})
            print("repeat", seed, name, round(reps[-4]["RMSE"], 3))
    pd.DataFrame(reps).to_csv(EVAL_DIR / "repeated_splits.csv", index=False)
    # ---- baselines on the main split (seed 42) saved for the evaluation script -------------------------
    for name, use_g in SETTINGS.items():
        pre, Xtr, Xval, Xte, ytr, yval, yte = prepare(df, use_g)
        for nm, mdl in {"ridge": Ridge(alpha=10.0), "forest": RandomForestRegressor(300, random_state=SEED, n_jobs=-1,
min_samples_leaf=2)}.items():
            mdl.fit(Xtr, ytr)
            if nm == "ridge": joblib.dump(mdl, MODEL_DIR / f"baseline_ridge_{name}.joblib")
    json.dump(summary, open(EVAL_DIR / "training_summary.json", "w"), indent=2)
    print(json.dumps(summary, indent=1)[:1500])


if __name__ == "__main__":
    run()
```

A.6 evaluate_model.py – evaluation, Figures 13–20

```
"""Evaluate both MLP settings on the held-out TEST set (seed-42 split): metrics, error analysis,
permutation importance and the repeated-split comparison. Writes Figures 13-20."""
import json, os
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from scipy import stats
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import Ridge
from sklearn.metrics import roc_auc_score
from config import MODEL_DIR, EVAL_DIR, SEED, PASS_MARK, BLUE, ORANGE, GREEN, set_style, save_fig
from data_preprocessing import load_data, clean_data, feature_columns
from train_model import prepare, predict, reg_metrics

TITLES = {"A": "Setting A: with G1 and G2", "B": "Setting B: without G1 and G2"}


def run():
    set_style(); df, _ = clean_data(load_data()); S = json.load(open(EVAL_DIR / "training_summary.json"))["settings"]
    res = {"metrics": {}, "passfail": {}}; P = {}; H = {}; imp_all = {}
    for name, use_g in {"A": True, "B": False}.items():
        pre, Xtr, Xval, Xte, ytr, yval, yte = prepare(df, use_g); cols = feature_columns(df, use_g)
        mlp = tf.keras.models.load_model(MODEL_DIR / f"mlp_setting_{name}.keras"); sc = tuple(S[name]["scale"]);
H[name] = json.load(open(EVAL_DIR / f"history_{name}.json"))
        p = predict(mlp, Xte, sc); P[name] = (yte, p)
        mm = {"MLP": reg_metrics(yte, p), "MLP (validation)": reg_metrics(yval, predict(mlp, Xval, sc)), "Mean
baseline": reg_metrics(yte, np.full_like(yte, ytr.mean(), dtype=float))}
        r = Ridge(alpha=10.0).fit(Xtr, ytr); mm["Ridge regression"] = reg_metrics(yte, np.clip(r.predict(Xte), 0, 20))
        f = RandomForestRegressor(300, random_state=SEED, n_jobs=-1, min_samples_leaf=2).fit(Xtr, ytr); mm["Random
forest"] = reg_metrics(yte, np.clip(f.predict(Xte), 0, 20))
        res["metrics"][name] = mm
        res["passfail"][name] = {"accuracy": float(((p >= PASS_MARK - 0.5) == (yte >= PASS_MARK)).mean()), "auc":
float(roc_auc_score(yte >= PASS_MARK, p)), "n_fail_test": int((yte < PASS_MARK).sum()), "n_test": int(len(yte))}
        # permutation importance on RAW columns: increase in test RMSE when a feature is shuffled
        rng = np.random.default_rng(SEED); Xraw = pd.DataFrame(df.drop(columns=["G3"]).iloc[0:0]); base = mm["MLP"]
["RMSE"]
        from data_preprocessing import split_data
        _, _, X_te_raw, _, _, y_te_raw = split_data(df); imp = {}
        for col in cols:
            d = []
            for _ in range(10):
                Xp = X_te_raw.copy(); Xp[col] = rng.permutation(Xp[col].values)
                d.append(reg_metrics(yte, predict(mlp, pre.transform(Xp[cols]).astype("float32"), sc))["RMSE"] - base)
            imp[col] = (np.mean(d), np.std(d))
```

```
        imp = pd.DataFrame(imp, index=["mean_increase", "std"]).T.sort_values("mean_increase", ascending=False);
imp.round(4).to_csv(EVAL_DIR / f"table_permutation_importance_{name}.csv"); imp_all[name] = imp
        res["residual_stats_" + name] = {"mean_error": float((p - yte).mean()), "sd_error": float((p -
yte).std(ddof=1)), "within_1": float((np.abs(p - yte) <= 1).mean()), "within_2": float((np.abs(p - yte) <= 2).mean()),
                                         "shapiro_p": float(stats.shapiro(p - yte)[1]), "max_abs_error": float(np.abs(p
- yte).max()), "worst_actual": float(yte[np.argmax(np.abs(p - yte))]), "worst_pred": float(p[np.argmax(np.abs(p -
yte))])}
    res["top_importance"] = {k: v.head(10).round(4).reset_index().rename(columns={"index":
"feature"}).to_dict("records") for k, v in imp_all.items()}
    pd.DataFrame({f"{n}|{k}": v for n, d in res["metrics"].items() for k, v in d.items()}).T.round(4).to_csv(EVAL_DIR /
"table_test_metrics.csv")
    # ---- Fig 13 / 14: learning curves ----------------------------------------------------------
    for key, ttl, yl, fn in [("loss", "Fig. 13  Training vs validation loss (MSE, standardised target)", "MSE loss",
"fig13_loss_vs_epoch.png"), ("mae", "Fig. 14  Training vs validation MAE (standardised target)", "MAE",
"fig14_mae_vs_epoch.png")]:
        fig, axes = plt.subplots(1, 2, figsize=(11, 3.9))
        for ax, n in zip(axes, "AB"):
            h = H[n]; ep = np.arange(1, len(h[key]) + 1); be = int(np.argmin(h["val_loss"])) + 1
            ax.plot(ep, h[key], color=BLUE, lw=2, label="Training"); ax.plot(ep, h["val_" + key], color=ORANGE, lw=2,
label="Validation"); ax.axvline(be, color="#52514e", ls=":", lw=1.2)
            ax.text(be + .5, ax.get_ylim()[1] * .55, f"best epoch {be}", fontsize=8); ax.set(title=TITLES[n],
xlabel="Epoch", ylabel=yl); ax.legend()
            res.setdefault("best_epoch", {})[n] = be; res.setdefault("epochs_run", {})[n] = len(ep)
        fig.suptitle(ttl, fontweight="bold", y=1.03); fig.tight_layout(); save_fig(fig, EVAL_DIR / fn)
    # ---- Fig 15: predicted vs actual -----------------------------------------------------------
    rng = np.random.default_rng(SEED); fig, axes = plt.subplots(1, 2, figsize=(10.4, 4.8), sharex=True, sharey=True)
    for ax, n in zip(axes, "AB"):
        y, p = P[n]; ax.scatter(y + rng.normal(0, .12, len(y)), p, s=22, alpha=.7, color=BLUE, edgecolor="none");
ax.plot([0, 20], [0, 20], color=ORANGE, lw=1.8, label="Perfect prediction")
        m = res["metrics"][n]["MLP"]; ax.set(title=f"{TITLES[n]}\nRMSE {m['RMSE']:.2f}, MAE {m['MAE']:.2f}, R²
{m['R2']:.2f}", xlabel="Actual G3", ylabel="Predicted G3" if n == "A" else "", xlim=(-1, 20.5), ylim=(-1, 20.5));
ax.legend(loc="upper left"); ax.set_aspect("equal")
    fig.suptitle("Fig. 15  Predicted vs actual final grade (test set)", fontweight="bold", y=1.02); fig.tight_layout();
save_fig(fig, EVAL_DIR / "fig15_predicted_vs_actual.png")
    # ---- Fig 16: error distribution ------------------------------------------------------------
    fig, axes = plt.subplots(1, 2, figsize=(10.4, 3.9), sharex=True)
    for ax, n in zip(axes, "AB"):
        y, p = P[n]; e = p - y; ax.hist(e, bins=np.arange(-12, 12.5, 1), color=BLUE, edgecolor="white", lw=.5);
ax.axvline(0, color="#0b0b0b", lw=1.3); ax.axvline(e.mean(), color=ORANGE, ls="--", lw=1.6, label=f"mean error
{e.mean():+.2f}")
        ax.set(title=f"{TITLES[n]}  (SD {e.std(ddof=1):.2f})", xlabel="Prediction error (predicted - actual)",
ylabel="Students" if n == "A" else ""); ax.legend()
    fig.suptitle("Fig. 16  Error distribution (test set)", fontweight="bold", y=1.03); fig.tight_layout();
save_fig(fig, EVAL_DIR / "fig16_error_distribution.png")
    # ---- Fig 17: residual vs predicted ---------------------------------------------------------
    fig, axes = plt.subplots(1, 2, figsize=(10.4, 3.9), sharey=True)
    for ax, n in zip(axes, "AB"):
        y, p = P[n]; ax.scatter(p, p - y, s=20, alpha=.7, color=BLUE, edgecolor="none"); ax.axhline(0, color=ORANGE,
lw=1.6); ax.set(title=TITLES[n], xlabel="Predicted G3", ylabel="Error (predicted - actual)" if n == "A" else "")
    fig.suptitle("Fig. 17  Residual plot (test set)", fontweight="bold", y=1.03); fig.tight_layout(); save_fig(fig,
EVAL_DIR / "fig17_residual_plot.png")
    # ---- Fig 18: permutation importance --------------------------------------------------------
    fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.6))
    for ax, n in zip(axes, "AB"):
        t = imp_all[n].head(10).iloc[::-1]; ax.barh(t.index, t.mean_increase, xerr=t["std"], color=BLUE, height=.65,
error_kw={"lw": 1, "ecolor": "#52514e"})
        ax.set(title=TITLES[n], xlabel="Increase in test RMSE when shuffled"); ax.grid(axis="y", visible=False)
    fig.suptitle("Fig. 18  Permutation importance (top 10 features)", fontweight="bold", y=1.02); fig.tight_layout();
save_fig(fig, EVAL_DIR / "fig18_permutation_importance.png")
    # ---- Fig 19: tuning ------------------------------------------------------------------------
    fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.8))
    for ax, n in zip(axes, "AB"):
        t = pd.read_csv(EVAL_DIR / f"tuning_results_{n}.csv"); t["cfg"] = t.hidden + " d=" + t.dropout.astype(str) + "
lr=" + t.lr.astype(str); t = t.sort_values("val_RMSE", ascending=False)
        ax.barh(t.cfg, t.val_RMSE, color=[ORANGE if v == t.val_RMSE.min() else BLUE for v in t.val_RMSE], height=.65)
        for y, v in enumerate(t.val_RMSE): ax.text(v + .01, y, f"{v:.2f}", va="center", fontsize=7)
        ax.set(title=TITLES[n], xlabel="Validation RMSE (lower is better)", xlim=(0, t.val_RMSE.max() * 1.12));
ax.grid(axis="y", visible=False); ax.tick_params(axis="y", labelsize=7)
    fig.suptitle("Fig. 19  Hyperparameter search results", fontweight="bold", y=1.02); fig.tight_layout();
save_fig(fig, EVAL_DIR / "fig19_tuning_results.png")
    # ---- Fig 20: repeated random splits
-----------------------------------------------------------------------------
    rp = pd.read_csv(EVAL_DIR / "repeated_splits.csv"); order = ["Mean baseline", "Ridge regression", "Random forest",
"MLP"]; cols = {"Mean baseline": "#9b9a93", "Ridge regression": GREEN, "Random forest": ORANGE, "MLP": BLUE}
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), sharey=True)
    for ax, n in zip(axes, "AB"):
        d = [rp[(rp.setting == n) & (rp.model == m)].RMSE.values for m in order]; bp = ax.boxplot(d, tick_labels=order,
patch_artist=True, widths=.55, medianprops=dict(color="#0b0b0b"))
        for b, m in zip(bp["boxes"], order): b.set(facecolor=cols[m], alpha=.85)
        ax.set(title=TITLES[n], ylabel="Test RMSE over 10 random splits" if n == "A" else ""); ax.tick_params(axis="x",
labelsize=8)
    fig.suptitle("Fig. 20  Stability over 10 random train/validation/test splits", fontweight="bold", y=1.02);
fig.tight_layout(); save_fig(fig, EVAL_DIR / "fig20_repeated_splits.png")
    sm = rp.groupby(["setting", "model"])[["MAE", "RMSE", "R2"]].agg(["mean", "std"]).round(4); sm.to_csv(EVAL_DIR /
"table_repeated_splits_summary.csv"); res["repeated"] = {f"{a}|{b}": {m: {"mean": float(sm.loc[(a, b), (m, "mean")]),
"std": float(sm.loc[(a, b), (m, "std")])} for m in ["MAE", "RMSE", "R2"]} for a, b in sm.index}
    json.dump(res, open(EVAL_DIR / "results_model.json", "w"), indent=1, default=float)
    print(json.dumps({k: res[k] for k in ["metrics", "passfail", "residual_stats_A", "residual_stats_B", "best_epoch",
"epochs_run"]}, indent=1, default=float)); print(sm); print(imp_all["A"].head(8)); print(imp_all["B"].head(8))


if __name__ == "__main__":
    run()
```

A.7 diagrams.py – workflow and architecture diagrams

```
"""Draw two explanatory diagrams: (1) project workflow, (2) MLP architecture."""
import json
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
from config import EVAL_DIR, EDA_DIR, set_style, save_fig, INK, INK2


def box(ax, x, y, w, h, text, fc="#e8f0fb", ec="#2a78d6", fs=9):
    ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.02,rounding_size=0.08", fc=fc, ec=ec, lw=1.3))
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center", fontsize=fs, color=INK)


def arrow(ax, x1, y1, x2, y2):
    ax.annotate("", xy=(x2, y2), xytext=(x1, y1), arrowprops=dict(arrowstyle="-|>", color=INK2, lw=1.3))


def workflow():
    fig, ax = plt.subplots(figsize=(10, 3.9)); ax.axis("off"); ax.set_xlim(0, 10); ax.set_ylim(0, 4)
    top = [("1. Acquire\nUCI dataset\n649 x 33", 0.1), ("2. Parse &\nclean\ntypes, duplicates,\nmissing values", 2.1),
("3. EDA &\nstatistics\nunivariate →\nmultivariate", 4.1),
           ("4. Visualise\n12 figures +\n2 interactive", 6.1), ("5. Split\n70 / 15 / 15\nstratified by\ngrade band",
8.1)]
    for t, x in top: box(ax, x, 2.3, 1.75, 1.35, t, fs=8.5)
    for i in range(4): arrow(ax, top[i][1] + 1.77, 2.97, top[i + 1][1] - 0.02, 2.97)
    bot = [("6. Preprocess (train-fit)\nwinsorise → scale,\none-hot, 0/1 flags", 0.1), ("7. Tune MLP\n12
configurations\nfor each setting", 2.65), ("8. Train final MLPs\nA: with G1, G2\nB: without G1, G2", 5.2), ("9.
Evaluate on test\nMAE, RMSE, R², errors,\nimportance, 10 splits", 7.75)]
    for t, x in bot: box(ax, x, 0.35, 2.1, 1.25, t, fc="#fdeee6", ec="#eb6834", fs=8.5)
    for i in range(3): arrow(ax, bot[i][1] + 2.12, 0.97, bot[i + 1][1] - 0.02, 0.97)
    arrow(ax, 8.97, 2.28, 1.15, 1.62); ax.text(5.0, 3.95, "Diagram 1  End-to-end project workflow", ha="center",
fontsize=12, fontweight="bold")
    save_fig(fig, EDA_DIR / "diagram1_workflow.png")


def architecture():
    s = json.load(open(EVAL_DIR / "training_summary.json"))["settings"]["A"]; hid = s["best_config"]["hidden"]; dr =
s["best_config"]["dropout"]
    fig, ax = plt.subplots(figsize=(10, 3.4)); ax.axis("off"); ax.set_xlim(0, 10); ax.set_ylim(0, 3.4)
    blocks = [(f"Input\n{s['n_inputs']} features", "#e8f0fb", "#2a78d6")] + [(f"Dense {h}\nReLU + L2\nBatchNorm\
nDropout {dr}", "#eaf6f0", "#1baf7a") for h in hid] + [("Dense 1\nlinear", "#fdeee6", "#eb6834"), ("Output\npredicted
G3\n(clipped to 0-20)", "#e8f0fb", "#2a78d6")]
    n = len(blocks); w = 1.5; gap = (10 - 0.3 - n * w) / (n - 1)
    for i, (t, fc, ec) in enumerate(blocks):
        x = 0.15 + i * (w + gap); box(ax, x, 0.7, w, 1.6, t, fc=fc, ec=ec, fs=8.5)
        if i < n - 1: arrow(ax, x + w + 0.02, 1.5, x + w + gap - 0.02, 1.5)
    ax.text(5, 3.0, f"Diagram 2  MLP architecture, setting A ({s['n_params']:,} parameters)", ha="center", fontsize=12,
fontweight="bold")
    save_fig(fig, EVAL_DIR / "diagram2_mlp_architecture.png")


if __name__ == "__main__":
    set_style(); workflow(); architecture()
```

A.8 run_all.py – run the whole pipeline

```
"""Run the whole pipeline end-to-end:  python src/run_all.py"""
import sys, pathlib
sys.path.insert(0, str(pathlib.Path(__file__).parent))
import eda, visualizations, train_model, evaluate_model, diagrams

if __name__ == "__main__":
    eda.run()                # tables + statistics
    visualizations.run()     # Figures 1-12 + interactive pages
    train_model.run()        # preprocessing, tuning (2 settings), baselines, 10 repeated splits
    evaluate_model.run()     # test metrics + Figures 13-20
    diagrams.set_style(); diagrams.workflow(); diagrams.architecture()
```

Appendix B – GitHub Repository and Checklist

B.1 Repository Structure

```
student-grade-prediction/
├── data/
│   └── student-por.csv              # UCI dataset 320, Portuguese course (649 x 33)
├── src/
│   ├── config.py                    # paths, seed, column groups, plot style
│   ├── data_preprocessing.py        # load, clean, split, leakage-safe pipeline
│   ├── eda.py                       # tables + statistical tests + distribution fits
│   ├── visualizations.py            # Figures 1-12 + interactive HTML pages
│   ├── train_model.py               # grid search, MLP training, baselines, 10 splits
│   ├── evaluate_model.py            # test metrics + Figures 13-20
│   ├── diagrams.py                  # workflow / architecture diagrams
│   └── run_all.py                   # runs the full pipeline
├── outputs/
│   ├── eda/                         # EDA figures, tables, interactive pages
│   └── model_evaluation/            # evaluation figures, metrics JSON/CSV
├── models/                          # saved MLPs (.keras) + preprocessors (.joblib)
├── report/                          # Word + PDF report, report builder scripts
├── requirements.txt
├── README.md
└── .gitignore
```

Table 23: Purpose of each folder and file

Path	Purpose
data/	The dataset used. CC BY 4.0 – keep the citation in the README
src/	All source code, one module per stage so it is easy to read and explain in a viva
outputs/eda, outputs/model_evaluation	Generated figures, tables and metrics, so the report can be checked without re-running
models/	Trained MLPs and fitted preprocessors (small files)
report/	Final report (DOCX and PDF) and the scripts that build it from the saved results
requirements.txt	Exact library versions used
README.md	Project summary, results, how to run
.gitignore	Keeps caches and secrets out of the repository

B.2 README Outline

- Title, one-paragraph summary and student details (RITHI R, 24AD091, U21ADP05)
- Problem statement and objectives
- Dataset: name, UCI URL, DOI, licence, size, target
- Project structure (tree above)
- How to run: pip install -r requirements.txt, then python src/run_all.py
- Key results table and 3–4 sample figures
- Key insights
- Limitations and future work
- References / citation of the dataset
- Link to the report in report/

B.3 Commands to Publish

```
cd student-grade-prediction
git init
git add .
git commit -m "Student grade prediction: EDA and MLP regression case study"
git branch -M main
git remote add origin https://github.com/<your-github-id>/student-grade-prediction.git
git push -u origin main
```

B.4 GitHub Submission Checklist

Table 24: Checklist before submitting the link

✓	Item
☐	Repository is **public** and created under your own GitHub ID
☐	Project report (PDF and Word) is inside report/
☐	Source code (.py) is inside src/ and runs with python src/run_all.py
☐	Dataset file is present (data/student-por.csv) and the UCI link is in the README
☐	README.md has summary, dataset, how to run, results and limitations
☐	No passwords, tokens, .env files or personal data are committed
☐	Public GitHub link pasted into the Google Form (https://forms.gle/kT9DU5aERo2peTNh8) or emailed to the coordinator

Appendix C – Final Checklist Against the Assignment Requirements

Table 25: Requirement checklist

Requirement from assignment sheet	Status	Where
Real-world dataset with ≥ 15 features	Done – 32 input features	2.1–2.4
Load data and perform EDA	Done	Ch. 3, 5
Handle missing values, outliers, duplicates	Done – none missing/duplicated; outliers analysed and winsorised	3.2, 3.3
Cleaning, transformation, encoding, scaling	Done	3.4
Train / validation / test split	Done – 70/15/15 stratified	3.6
Minimum 5 visualisations covering all 7 stages	Done – 12 EDA + 8 model figures; stages mapped in Table 9	4.1, 4.4
Mention what each visual shows, why created, insights	Done – explanation table per figure	4.4, 7.3
At least one deep-learning model	Done – MLP regressor (two settings)	Ch. 6
Feature analysis, importance, basic tuning	Done	6.5, 6.6
3–4 result visualisations (loss, accuracy/MAE, error distribution, predicted vs actual)	Done – Figs. 13–20 (regression task, so no confusion matrix/ROC)	7.3
Conclusion, challenges, future scope	Done	Ch. 8
3–5 references, APA/IEEE	Done – 14 APA references	9.2
Report format (title page … appendix)	Done	Whole report
Well-commented code covering loading, EDA, preprocessing, model, evaluation	Done	Appendix A
GitHub repository, README, dataset, report, source code	Prepared locally; **upload and public link still to be done by the student**	Appendix B